

252-0027

Einführung in die Programmierung

Übungen

Woche 11: Vererbung

Joran Bühler

Departement Informatik

ETH Zürich

Ablauf

- Nachbesprechung
- Scanner
- Vererbung (recap)
- Prüfungsaufgabe HS 21
- Vorbesprechung
- (Evtl Loop invariante)

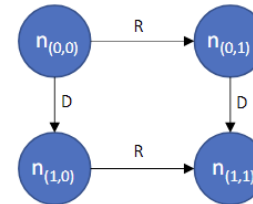
Nachbesprechung

Aufgabe 1: Square Grid

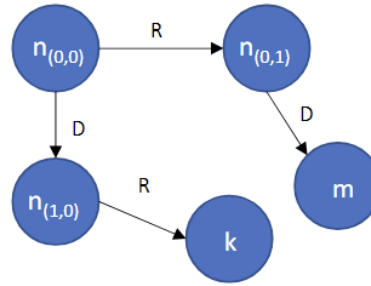
In dieser Aufgabe betrachten wir gerichtete Graphen, wobei es für jeden Knoten g höchstens zwei gerichtete Kanten von g zu anderen Knoten f, h geben kann (f, g, h können gleich sein). Wir unterscheiden dabei zwischen der rechten und der unteren Kante (und damit dem rechten und dem unteren Knoten).

Die Klasse `Node` repräsentiert einen Knoten in einem solchen Graphen. Die Methode `Node.getRight()` (bzw. `Node.getDown()`) gibt den rechten Knoten (bzw. unteren Knoten) zurück (als `Node`-Objekt). Wenn der rechte Knoten von n_0 nicht existiert, dann gibt `Node.getRight()` `null` zurück (analog für den unteren Knoten). Die Methode `Node.setRight(Node r)` (bzw. `Node.setDown(Node d)`) setzt den rechten (bzw. unteren) Knoten.

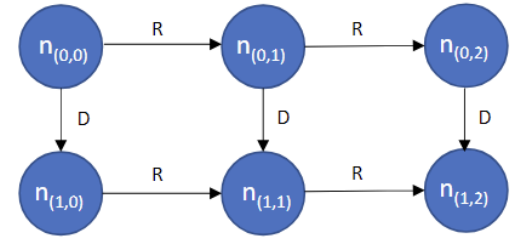
Das Ziel der Aufgabe ist, einen von einem `Node`-Objekt definierten Graphen zu analysieren. Konkret geht es darum, die Grösse des grössten quadratischen Gitters in dem Graphen zu bestimmen, der mit dem übergebenen `Node`-Objekt beschrieben wird, welches den gleichen Ursprungsknoten wie der Graph hat.



Aufgabe 1: Square Grid



(a)



(b)

Abbildung 2: Graphen mit quadratischen Gittern als Teilgraphen

Referenzen vs Objekte

Aufgabe 2: Umkehrung

In einem vorherigen Übungsblatt haben Sie eine Linked List für Integers implementiert. In dieser Aufgabe fügen Sie dieser `LinkedList` eine weitere Methode hinzu, welche die Liste umkehrt. Eine Liste gilt als umgekehrt, wenn für jedes Paar von Nodes `a` und `b`, für welche zuvor `a == b.next` gegolten hat, in der neuen (umgekehrten) Liste `b == a.next` gilt. Zusätzlich entspricht nach der Umkehrung der erste Node der neuen Liste dem letzten Node der ursprünglichen Liste (und umgekehrt).

Vervollständigen Sie die Methode `reverse()` in der Klasse `LinkedList`. Die Methode soll, wie oben definiert, die Liste umkehren. Achten Sie darauf, dass Sie wirklich die Reihenfolge der Nodes selbst umkehren. Es reicht nicht aus, die Reihenfolge der enthaltenen `int`-Werte umzukehren. Es müssen auch in der umgekehrten Liste dieselben Instanzen von `IntNodes` wie in der ursprünglichen Liste verwendet werden. Erstellen Sie also *keine* neuen `IntNodes` mit `new IntNode()`. In der Datei `“UmkehrungTest.java”` finden Sie einen einfachen Test.

Aufgabe 3: “KI” für das Ratespiel

In Übung 5 implementierten Sie ein Spiel, in welchem der Computer ein Wort auswählt und der Spieler dieses erraten muss. Dort war der Spieler der Benutzer des Programms. In dieser Aufgabe sollen Sie verschiedene “künstliche” Spieler entwickeln. Das heisst, anstelle des Menschen, der über die Konsole Tipps eingibt, werden die Tipps von (mehr oder weniger “intelligenten”) Programmen abgegeben. Ihr Ziel ist es, einen künstlichen Spieler zu entwickeln, der über mehrere Spiele hinweg die Wörter in so wenig Versuchen wie möglich errät.

Die Übungsvorlage enthält bereits den Code für das Ratespiel. Gegenüber Übung 5 ist dieser nun in verschiedene Klassen aufgeteilt. Die drei Hauptklassen sind `RateSpiel`, `Computer` und `Spieler`. Die Klasse `RateSpielApp` enthält eine `main`-Methode, welche das Spiel aufsetzt und durchführt. Durch die Aufteilung ist es möglich, mittels Vererbung Spieler mit unterschiedlichem Verhalten zu schreiben. Die Klasse `Spieler` enthält nämlich nur die Deklarationen der benötigten Methoden, aber keine (sinnvolle) Funktionalität. Subklassen von `Spieler` überschreiben diese Methoden und definieren damit das Verhalten eines Spielers.

Ein konkreter Spieler ist ebenfalls schon in der Vorlage vorhanden: der `KonsolenSpieler`. Dieser besitzt allerdings keine eigene “Intelligenz”, sondern holt sich die Tipps über die Konsole vom Benutzer. Ein `RateSpiel` mit einem `KonsolenSpieler` verhält sich also so wie das Spiel in Übung 5. Starten Sie die `RateSpielApp` und überzeugen Sie sich selbst¹.

Aufgabe 4: Klassenrätsel

In dieser Aufgabe sollen Sie zeigen, dass Sie mit Klassen und Vererbung umgehen können. Im Anhang A finden Sie ein Programm, welches Instanzen von Klassen erstellt und Methoden aufruft. Das Programm macht nichts Sinnvolles und dient nur dem Testen Ihrer Fähigkeiten. In Anhang B befinden sich die verwendeten Klassen, jedoch sind die Klassen noch nicht vollständig. Bei manchen der Klassen fehlt noch die `extends`-Klausel, welche angibt, dass eine Klasse von einer anderen Klasse erbt. Ihre Aufgabe ist es, die nötigen `extends`-Klauseln hinzuzufügen, so dass alles kompiliert und so dass die Ausgabe des Programms von Anhang A am Ende so aussieht wie im Anhang C gezeigt.

Der Code von Anhang A und Anhang B befindet sich in Ihrem `src`-Ordner. Zusätzlich enthält "KlassenTest.java" einen Unit-Test, welcher prüft, ob die Ausgabe des Programms dem Output aus Anhang C entspricht. Beachten Sie, dass Sie für diese Aufgabe **ausschliesslich** `extends`-Klauseln hinzufügen (diese kann es nur an den grauen Boxen aus Anhang B geben), kein anderer Code darf verändert werden.

Tipp: Lösen Sie die Aufgabe zuerst auf Papier, ohne die Hilfe von Eclipse. Sobald Sie herausgefunden haben, welche Klassen von welchen Klassen erben, testen Sie Ihre Lösung in Eclipse. Dies hilft Ihnen, Ihr Wissen über Vererbung zu testen. In der Vergangenheit wurden ähnliche Aufgaben im schriftlichen Teil der Prüfung gestellt.

Scanner

Scanner: Methoden

Scanner

- **next():** Wenn ein nächster String existiert im Scanner, dann wird dieser eingelesen und zurückgegeben. Sonst gibt es eine `NoSuchElementException`.
- **nextInt():** Wenn der nächste Token ein `Int` ist, dann wird dieser eingelesen und zurückgegeben. Sonst gibt es eine `NoSuchElementException`.
- **nextBoolean():** Wenn der nächste Token ein `Boolean` ist, dann wird dieser eingelesen und zurückgegeben. Sonst gibt es eine `NoSuchElementException`.
- **nextDouble():** Wenn der nächste Token ein `Double` ist, dann wird dieser eingelesen und zurückgegeben. Sonst gibt es eine `NoSuchElementException`.

Scanner: Methoden

Scanner

- `hasNext()`: Prüft ob der nächste Token im Scanner ein `String` ist.
- `hasNextInt()`: Prüft ob der nächste Token im Scanner ein `Int` ist.
- `hasNextBoolean()`: Prüft ob der nächste Token im Scanner ein `Boolean` ist.
- `hasNextDouble()`: Prüft ob der nächste Token im Scanner ein `Double` ist.

Scanner: LOCALE

Scanner

- Beim Einlesen von double kommt es zu Problemen...

```
jshell> scanner.nextDouble()  
| Exception java.util.InputMismatchException  
|     at Scanner.throwFor (Scanner.java:947)  
|     at Scanner.next (Scanner.java:1602)  
|     at Scanner.nextDouble (Scanner.java:2573)  
|     at (#3:1)
```

- Im Deutschen werden Kommazahlen mit , statt mit . benutzt. Deshalb kann ein Scanner nicht 4.5 einlesen.
- **Lösung:** scanner.useLocale(Locale.US).

Aufgabe 4: Rechnungen (erweitert)

In dieser Aufgabe erweitern Sie eine vorherige Aufgabe, in welcher ein System für Stromverbräuche Rechnungen erstellt. Konkret gibt es drei Erweiterungen: (1) Es sollen auch nicht korrekt formatierte Eingabedateien gehandhabt werden. (2) Ein Kunde kann eine beliebige Anzahl von Verbrauchswerten haben. (3) Es gibt eine neue Unteraufgabe b. In der folgenden Aufgabenbeschreibung für Unteraufgabe a sind die Änderung in **bold** markiert.

- a) Vervollständigen Sie die `process`-Methode in der Klasse `Bills`. Die Methode hat zwei Argumente: einen `Scanner`, von dem Sie den Inhalt der Eingabedatei lesen sollen, und einen `PrintStream`, in welchen Sie die unten beschriebenen Informationen schreiben.

Ihr Programm muss **auch mit manchen nicht korrekt formatierten Eingabedateien umgehen. Die Aufgabestellung gibt an, wie mit nicht korrekt formatierten Eingaben umzugehen ist.** Ein Beispiel einer korrekt formatierten Datei finden Sie im Projekt unter dem Namen "Data.txt". Exceptions im Zusammenhang mit Ein- und Ausgabe können Sie ignorieren.

Eine valide Eingabedatei enthält Zeilen, die entweder den Tarif, der angewendet werden soll, oder die Daten für den Stromverbrauch eines Kunden beschreiben. Der Verbrauch eines Kunden ist niemals grösser als 100000 Kilowattstunden.

Eine Tarifbeschreibung hat folgendes Format:

$$\text{Tarif_}n_l_1_p_1 \dots l_n_p_n$$

Aufgabe 4: Rechnungen (erweitert)

Ein einmal gelesener Tarif wird für alle Kunden angewendet, die nach dieser Tarifinformation in der Eingabedatei erscheinen. Wenn ein neuer Tarif erscheint, dann gilt der danach für die weiteren Kunden bis auf Weiteres. Sie können davon ausgehen, dass eine Kunden-ID nur einmal in der Eingabedatei vorkommen kann und dass die erste Zeile der Eingabedatei eine Tarifbeschreibung ist. Implementieren Sie die **Ihr Program soll Zeilen im Input ignorieren, welche weder mit "Tarif" noch einem positiven Integer beginnen. Falls ein Verbrauchswert im Input kein positiver Integer ist, dann soll eine `IllegalFormatException` geworfen werden. Die `IllegalFormatException` wird zur Verfügung gestellt.**

```
public void process(Scanner input, PrintStream output) {  
    while(input.hasNextLine()) {  
        //one possible approach  
        //String[] line = input.nextLine().split("");  
  
        //my approach  
        String lineString = input.nextLine();  
        Scanner line = new Scanner(lineString);  
        if(!line.hasNext()) {  
            continue;  
        }  
        String first = line.next();  
        if(isPositiveInt(first)) {  
            //TODO  
        }  
        else {  
            //TODO  
        }  
    }  
  
    for(Customer customer : customers) {  
        output.println(customer); //This assumes you overwrote toString()  
    }  
}
```

```
static private boolean isPositiveInt(String string) {  
    try {  
        int parsed = Integer.valueOf(string);  
        return parsed >= 0;  
    } catch (Exception e) {  
        return false;  
    }  
}
```

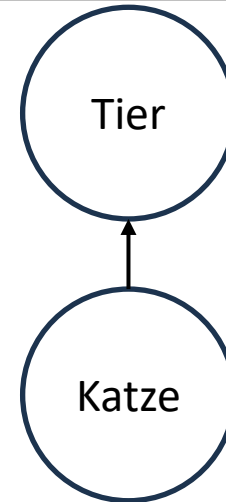
Inheritance

Extends-Schlüsselwort

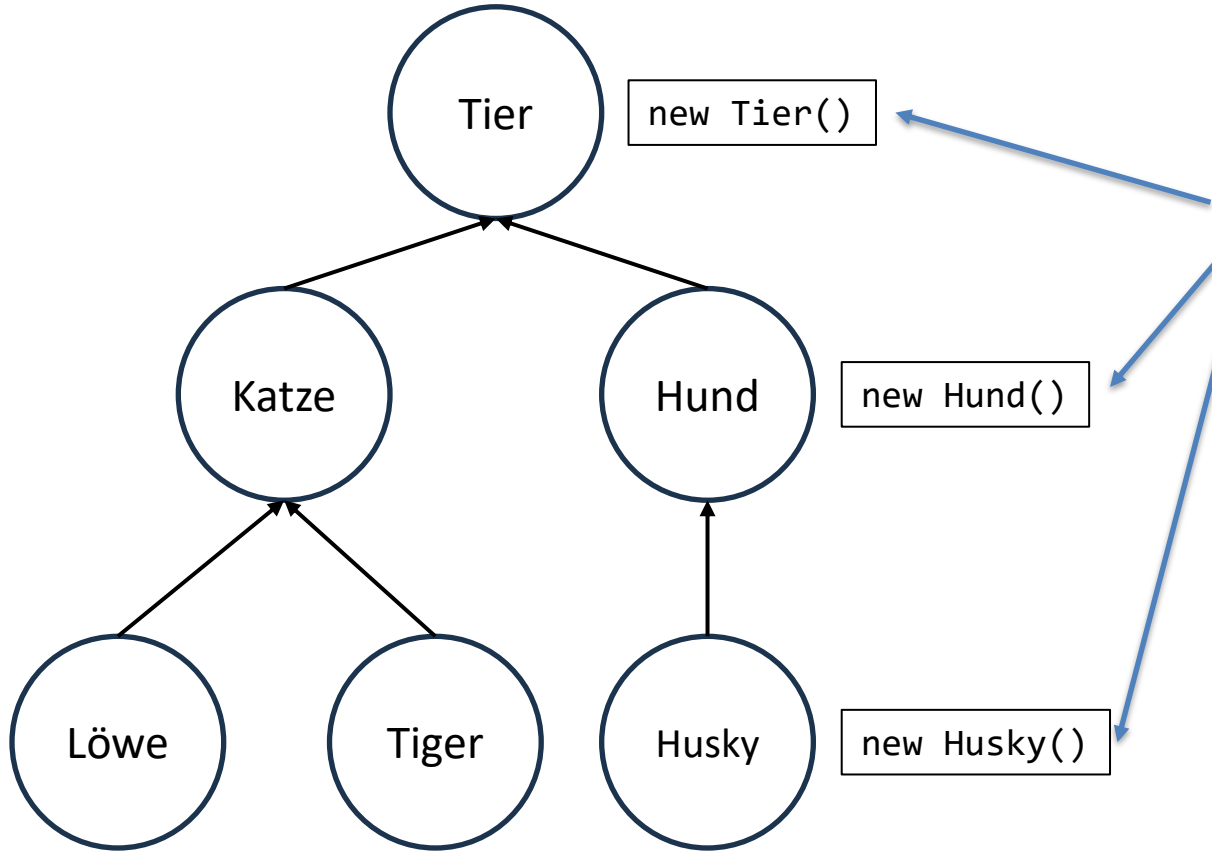
- **extends** spezifiziert, dass eine Klasse von einer anderen **erbt**
- Wir nennen die erbende Klasse im Folgenden Subklasse...
- und die vererbende Klasse Superklasse



```
1 public class Katze extends Tier{}
```



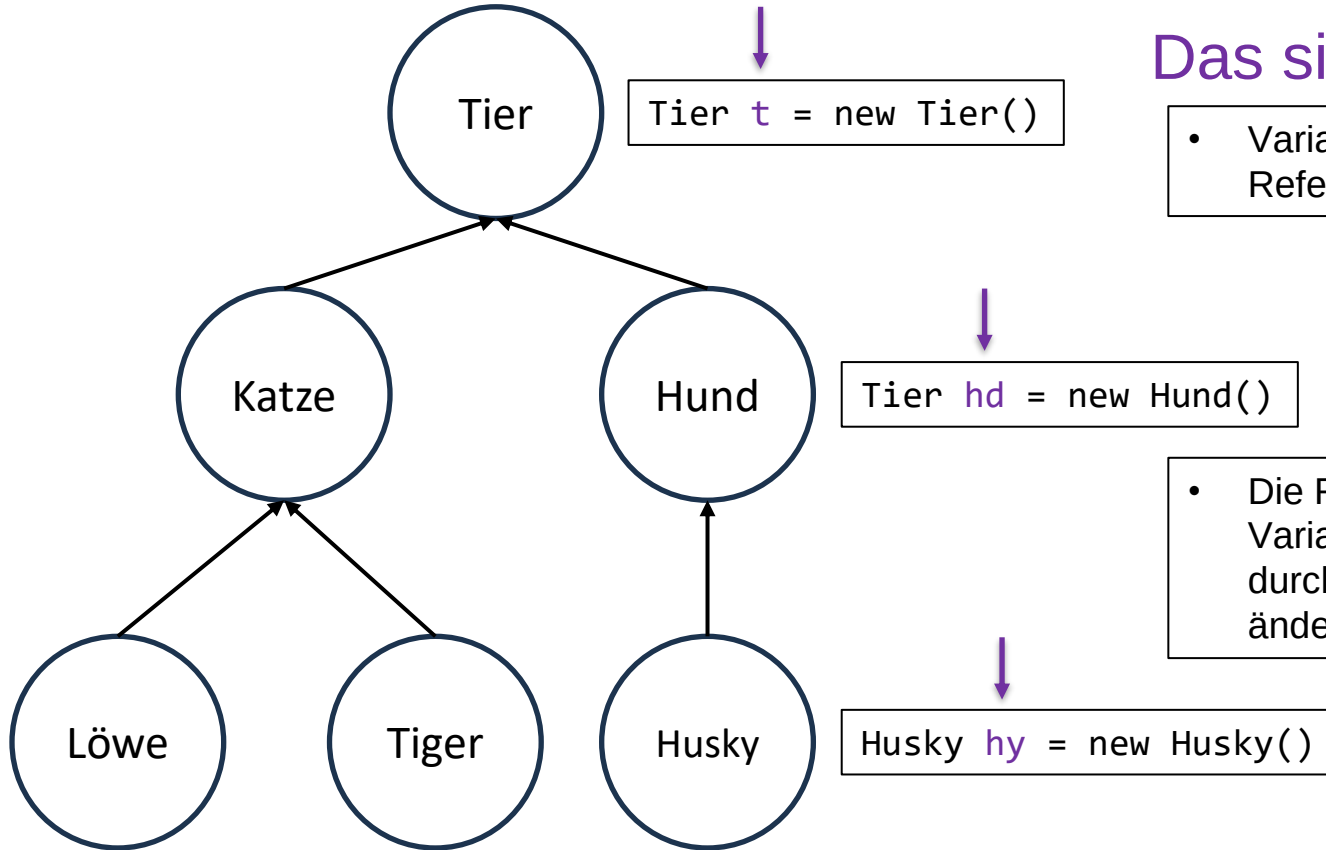
Objekt vs Referenz



Das sind Objekte.

- Objekte ändern **nie** ihren Typ.
- Ein Husky-Objekt ist und bleibt ein Husky-Objekt.
- Ein Katzen-Objekt kann kein Löwen-Objekt werden.

Objekt vs Referenz



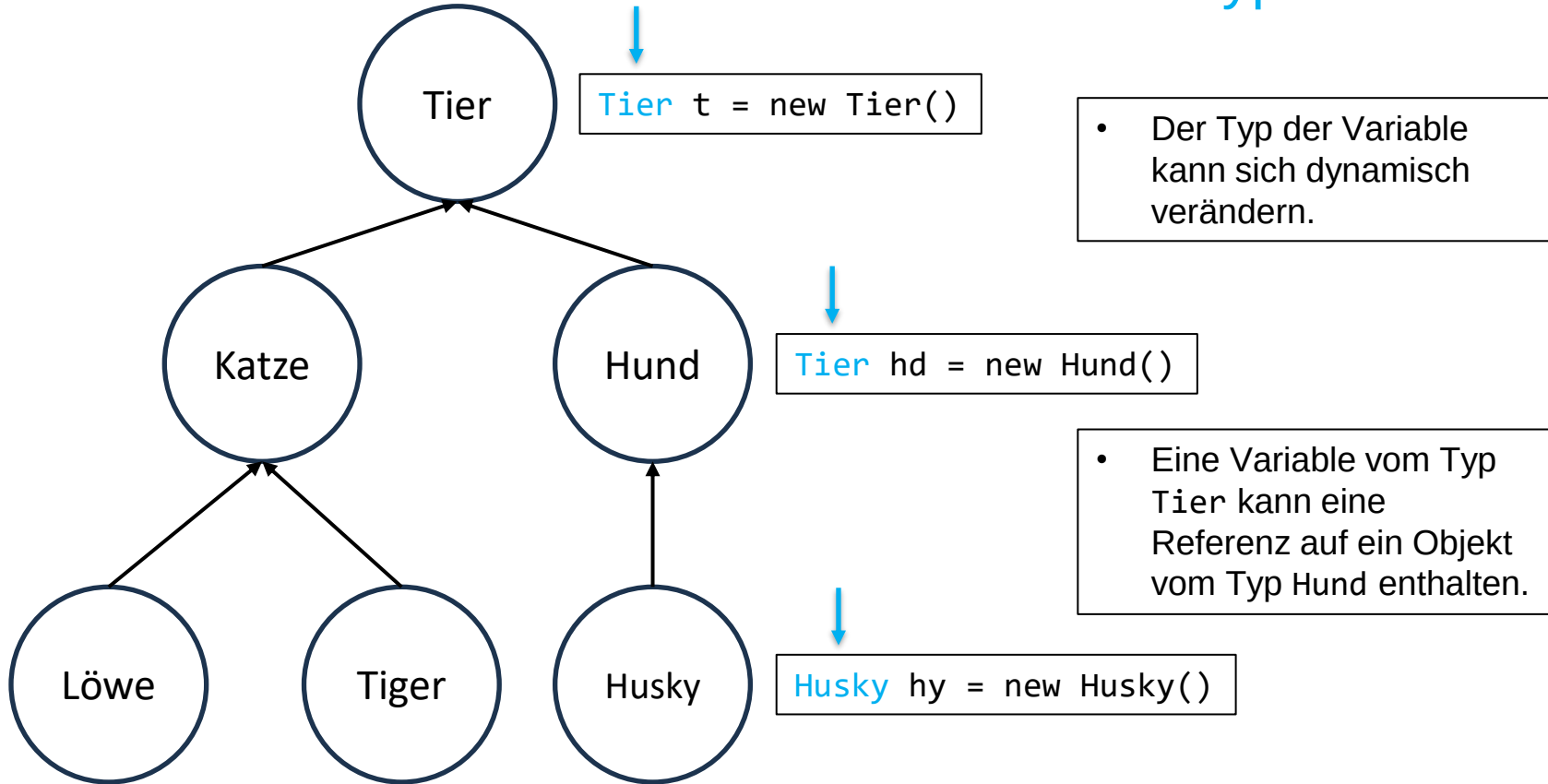
Das sind Variablen.

- Variablen enthalten Referenzen oder Werte.

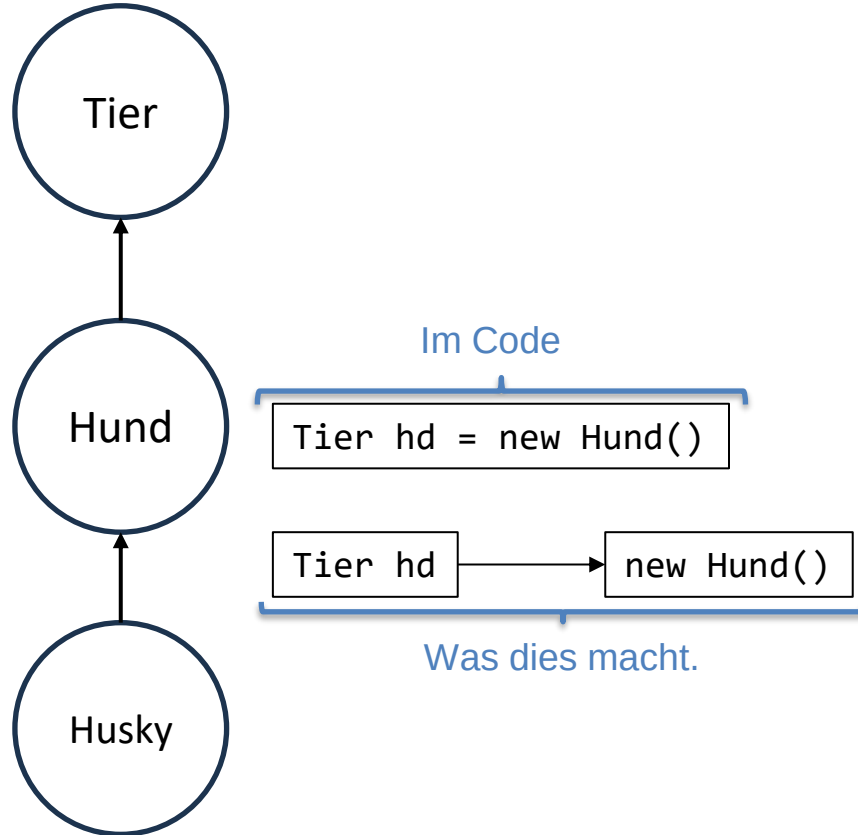
- Die Referenz in einer Variable kann sich durch Zuweisung mit = ändern.

Objekt vs Referenz

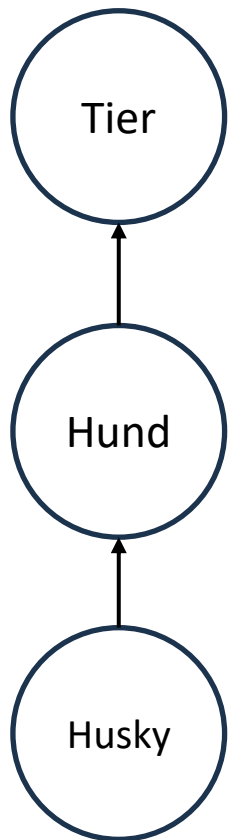
Das ist der Typ der Variable.



Objekt vs Referenz



Objekt vs Referenz

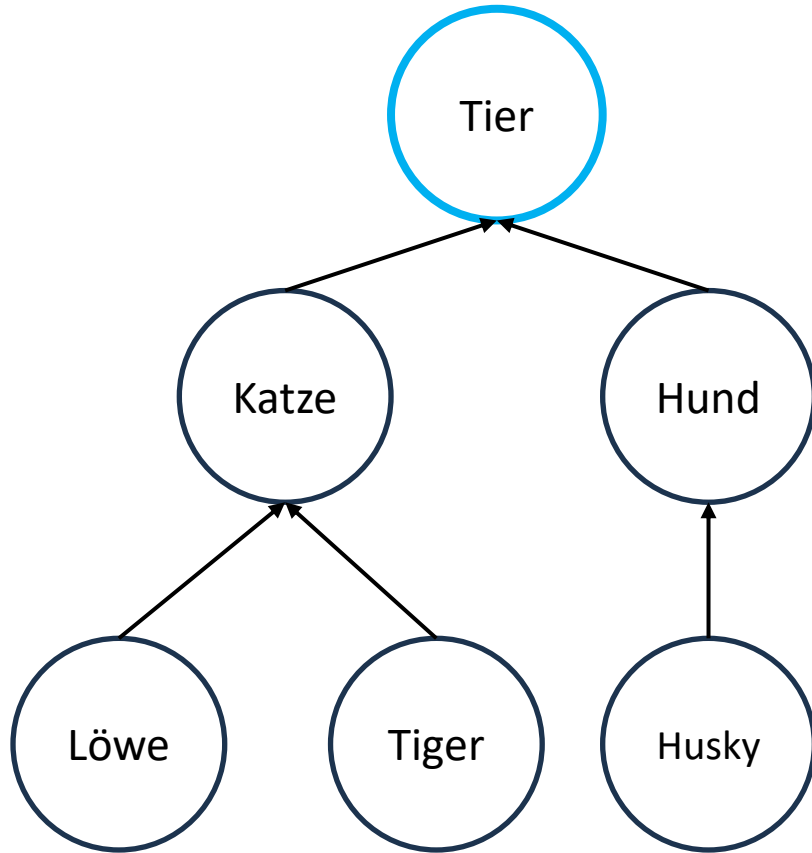


Wie unterscheiden wir zwischen:

- **Typ der Variable** (Statischer Typ)
- **Typ des Objekts** auf welches die **Referenz** in der Variable **zeigt?** (Dynamischer Typ)

```
Tier hd = new Husky()
```

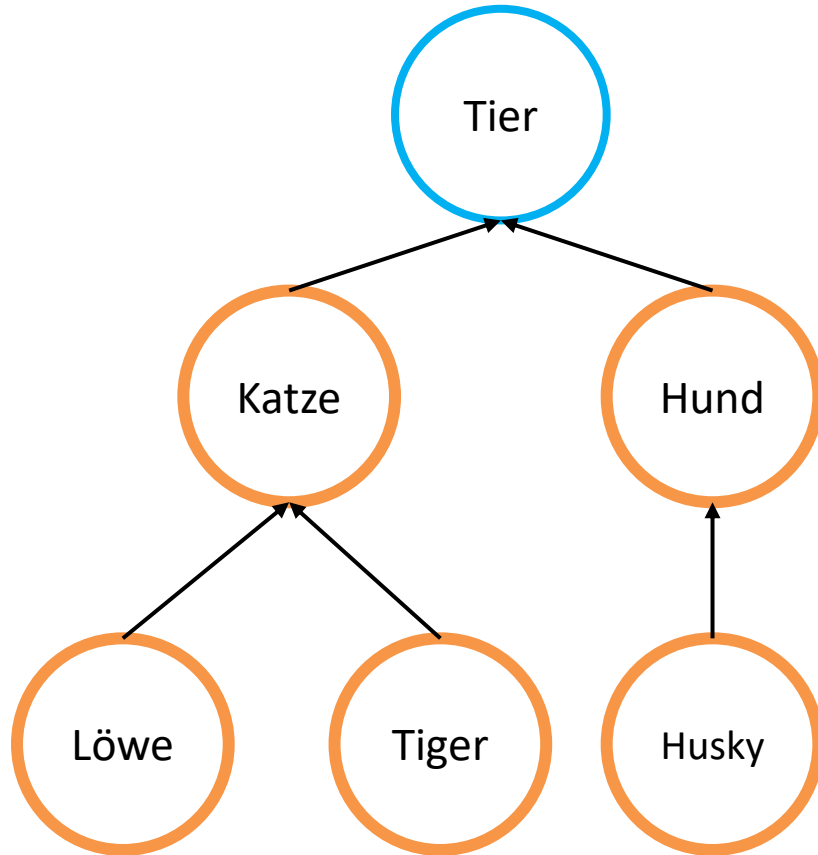
Was für Typen akzeptiert eine Variable?



Statischer Typ

Möglicher Dynamischer Typ

Verdeutlicht – Was für Typen akzeptiert eine Variable?



Funktionieren diese Zuweisungen?

```
Tier hd = new Husky()
```



```
Tier k1 = new Katze()
```



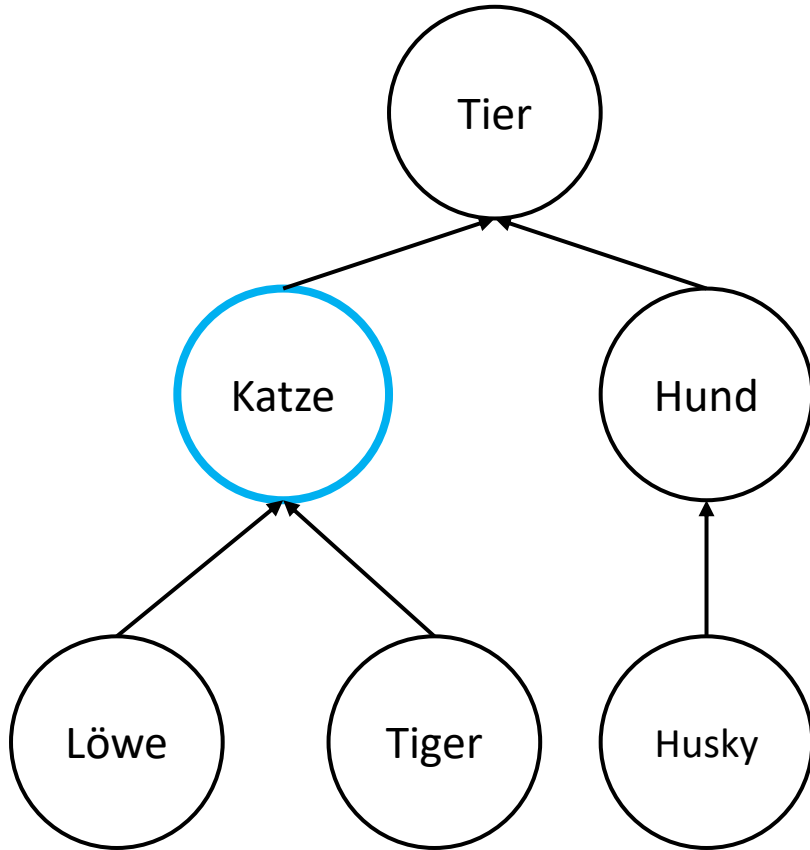
```
Tier t1 = new Tiger()
```



Statischer Typ

Möglicher Dynamischer Typ

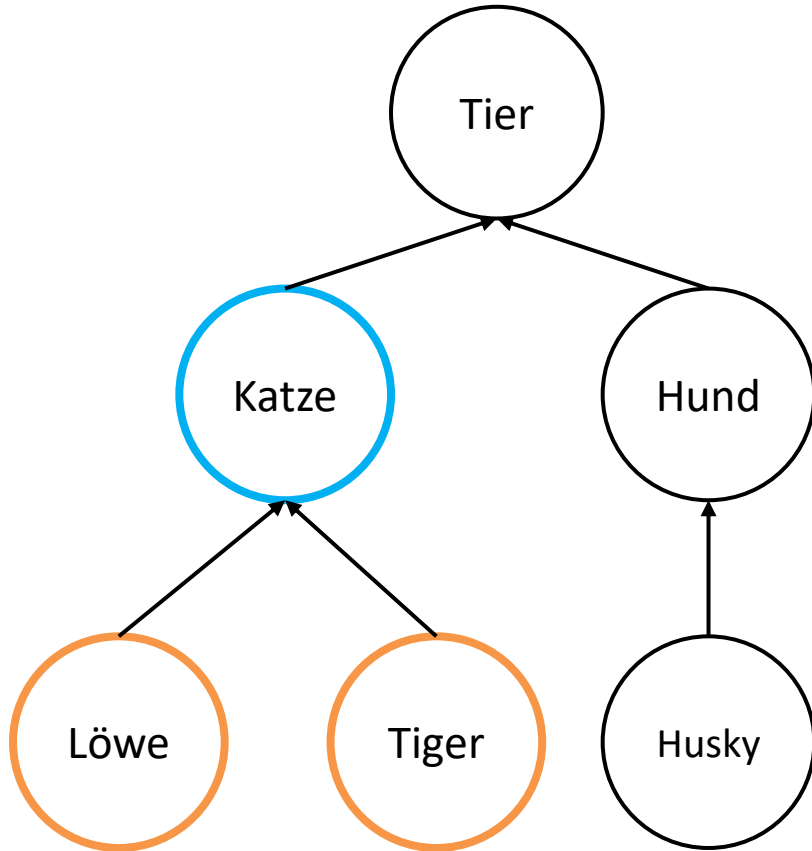
Verdeutlicht – Was für Typen akzeptiert eine Variable?



Statischer Typ

Möglicher Dynamischer Typ

Verdeutlicht – Was für Typen akzeptiert eine Variable?



Funktionieren diese Zuweisungen?

`Katze k1 = new Katze()`



`Tier t1 = new Tiger()`



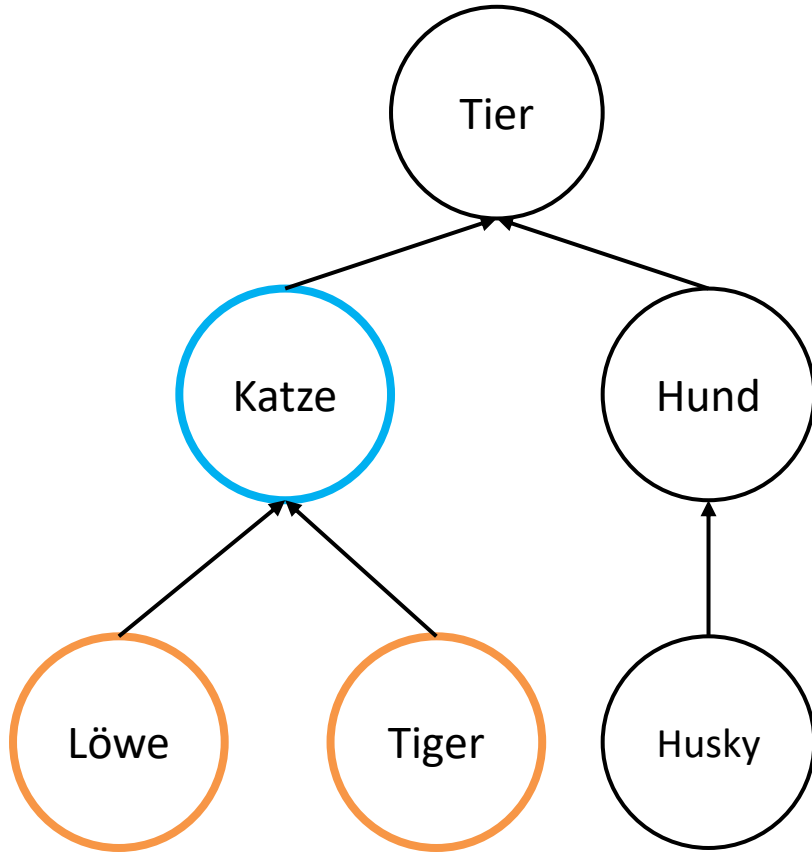
`Katze k3 = t1`



Statischer Typ

Möglicher Dynamischer Typ

Verdeutlicht – Was für Typen akzeptiert eine Variable?



Warum ist das problematisch?

```
Tier t1 = new Tiger()
```



```
Katze k3 = t1
```



Was sieht der Compiler?



```
Katze = Tier
```

Statischer Typ

Möglicher Dynamischer Typ

Verdeutlicht – Was für Typen akzeptiert eine Variable?

Warum ist das problematisch?

```
Tier t1 = new Tiger()
```



```
Katze k3 = t1
```



Was sieht der Compiler? 

```
Katze = Tier
```

Die Zuweisung ist unsicher, da der Compiler **keine Garantie** hat, dass der dynamische Typ von **t1** mit dem Typ **Katze** kompatibel ist.

Intuitiv: Nicht jedes Tier ist eine Katze.

Verdeutlicht – Was für Typen akzeptiert eine Variable?

Warum ist das problematisch?

```
Tier t1 = new Tiger()
```



```
Katze k3 = t1
```



Was sieht der Compiler? 

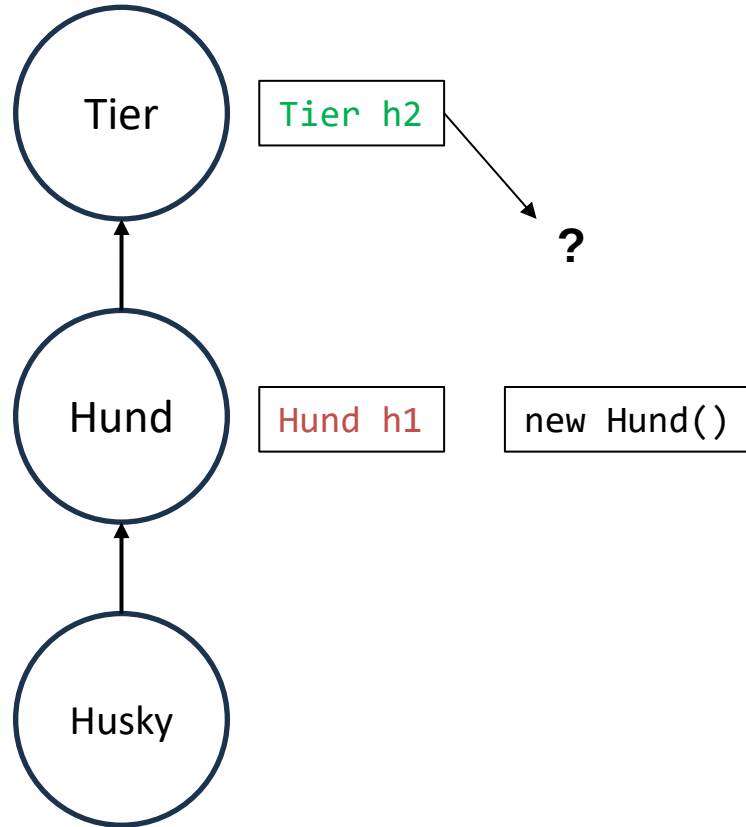
```
Katze = Tier
```

Aber wir probieren doch nur einen **Tiger** (subklasse der **Katze**) in **k3** zu speichern...

Wir müssen dem Compiler garantieren können, dass er hier immer eine Subklasse von Katze bekommt, damit die Zuweisung legal ist.

Casting

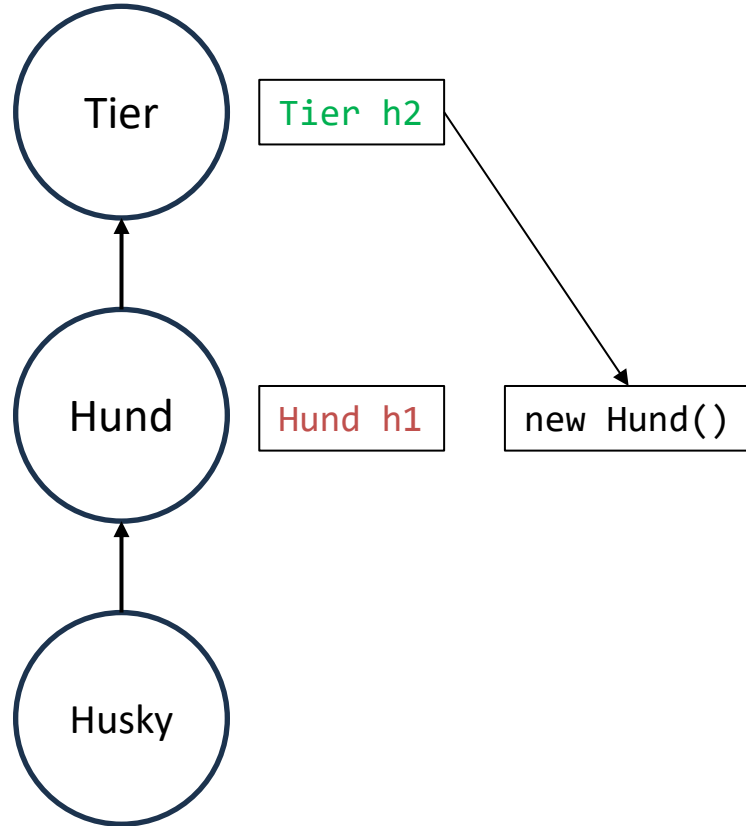
Casting



```
void methode1(Tier h2) {  
    Hund h1;  
    h1 = h2;   
}
```

Hier kennen wir den dynamischen Typ nicht...

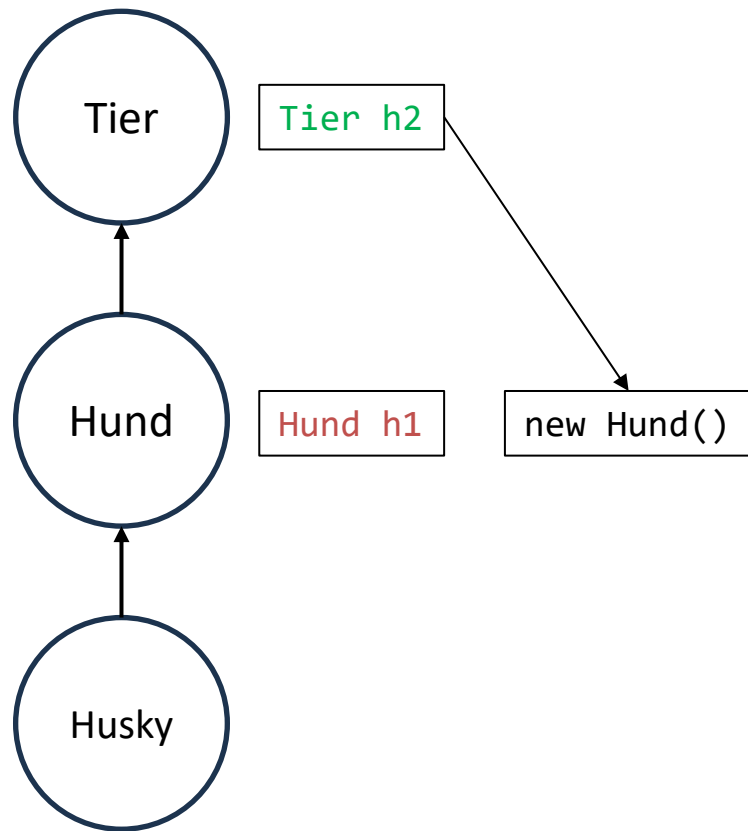
Casting



```
void methode1(Tier h2) {  
    Hund h1;  
    h1 = h2;   
}
```

Wenn wir methode1 nur aufrufen, wenn h2 eine Referenz auf ein Objekt vom Typ Hund enthält, dann würde eigentlich `h1 = h2` immer gehen!

Casting

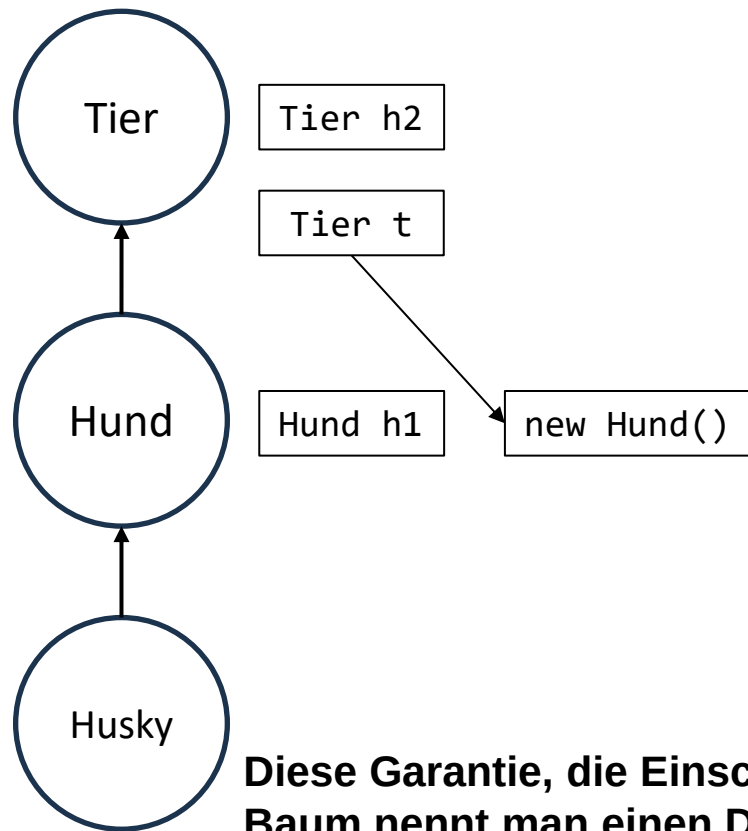


```
void methode1(Tier h2) {  
  
    Hund h1;  
  
    h1 = (Hund) h2;  
}
```

Wenn wir `methode1` nur aufrufen, wenn `h2` eine Referenz auf ein Objekt vom Typ `Hund` enthält, dann würde eigentlich `h1 = h2` immer gehen!

Ein Cast ist ein Versprechen an den Compiler, dass dies der Fall ist.

Casting



```
void methode1(Tier h2) {  
    Hund h1;  
    h1 = (Hund) h2;  
}
```

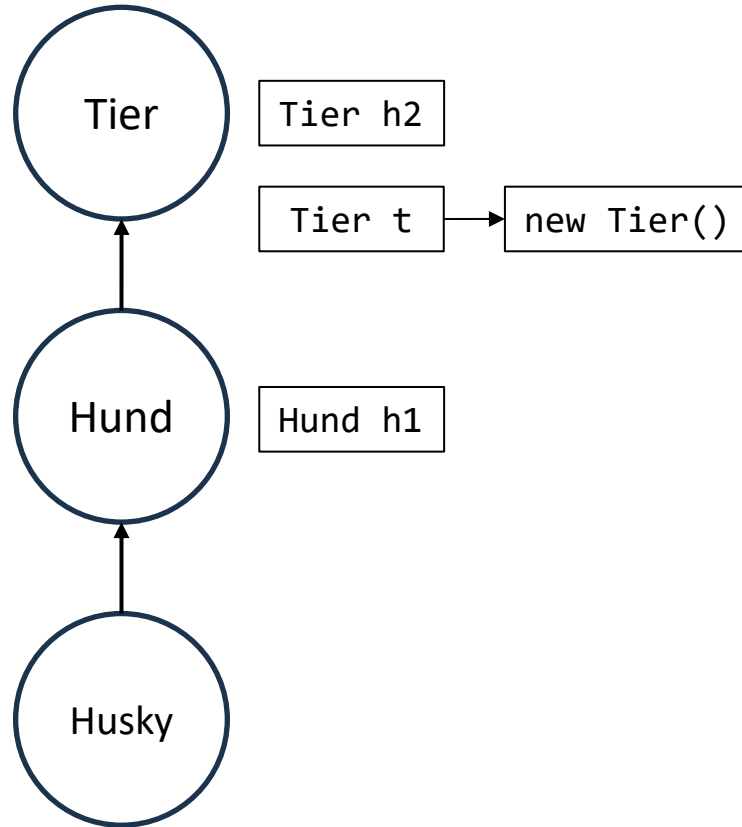
```
void methode2() {  
    Tier t = new Hund();  
    methode1(t);  
}
```

Geht das?



Diese Garantie, die Einschränkung auf einen typen weiter unten im Baum nennt man einen Downcast.

Casting

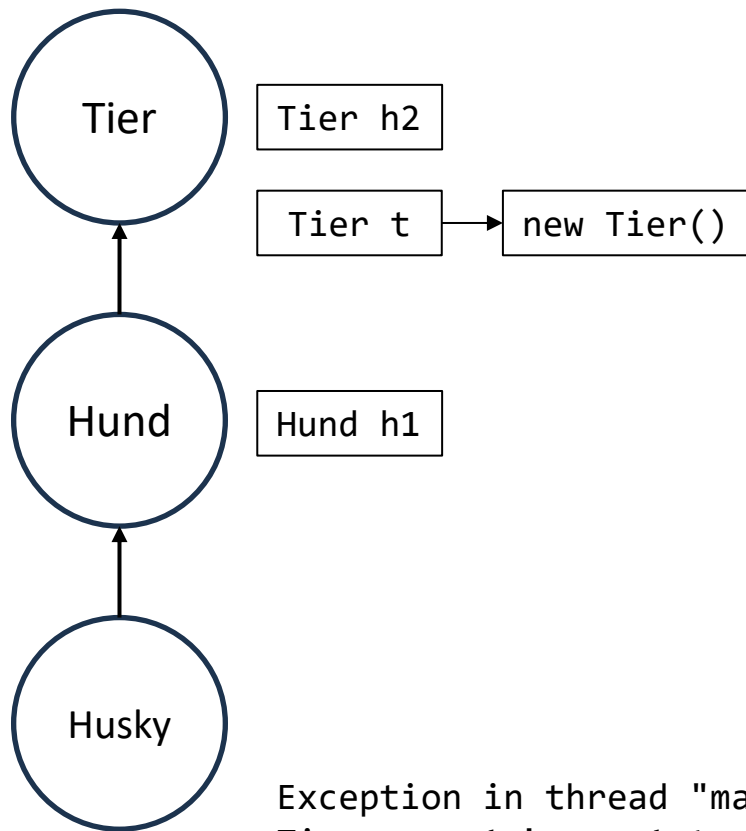


```
void methode1(Tier h2) {  
    Hund h1;  
    h1 = (Hund) h2;  
}
```

```
void methode2() {  
    Tier t = new Tier();  
    methode1(t);  
}
```

Geht das?

Casting



```
void methode1(Tier h2) {  
    Hund h1;  
    h1 = (Hund) h2;  
}
```

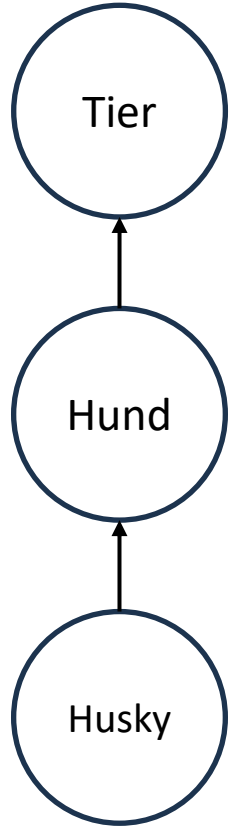


```
void methode2() {  
    Tier t = new Tier();  
    methode1(t);  
}
```

Geht das?

Exception in thread "main" java.lang.ClassCastException: class Tier **cannot be cast** to class Hund

Casting: Laufzeitfehler vs Compiler Fehler



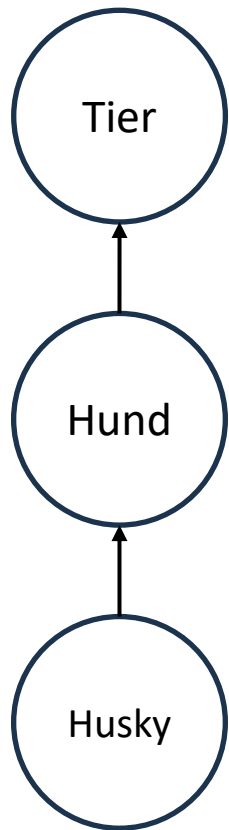
```
Hund h = new Tier();
```



```
Exception in thread "main" java.lang.Error: Unresolved  
compilation problem:  
    Type mismatch: cannot convert from Tier to Hund
```

Compiler-Fehler: Die Typen sind **nie** kompatibel.

Casting: Laufzeitfehler vs Compiler Fehler



```
Tier t = new Tier();  
Hund h = (Hund) t;
```



```
Exception in thread "main" java.lang.ClassCastException:  
class Tier cannot be cast to class Hund
```

Laufzeitfehler: Die Typen sind zwar nie kompatibel, aber das Versprechen (der Cast) an den Compiler lässt das Programm kompilieren.

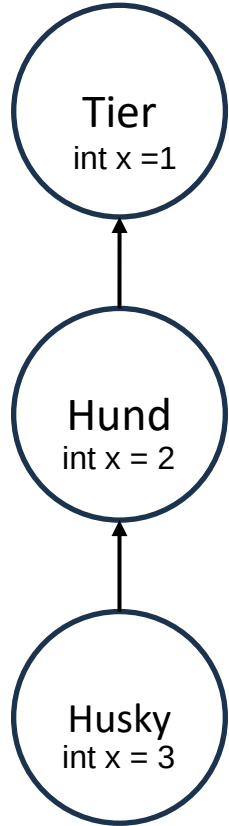
- Beim Ausführen gibt es einen Laufzeitfehler.

Type-Casting informell

- Wir betrachten die Vererbung als Baum
- Casting nach oben geht immer! (Upcasting)
- Casting nach unten geht, falls der Dynamische Typ von derselben Klasse oder einer Subklasse des Statischen Typs ist.
-> muss explizit gemacht werden.

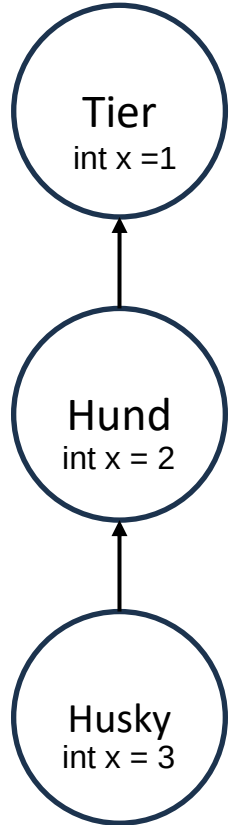
Attributwahl

Attribute:



Regel: Attribute werden an Hand vom **statischen Typ** ausgewählt.

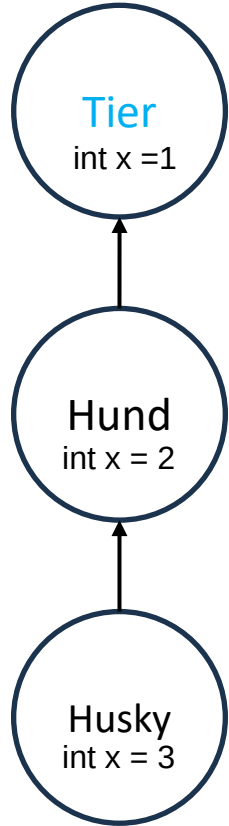
Attribute:



Regel: Attribute werden an Hand vom **statischen Typ** ausgewählt.

```
Tier t = new Tier();  
  
System.out.println(t.x);
```

Attribute:

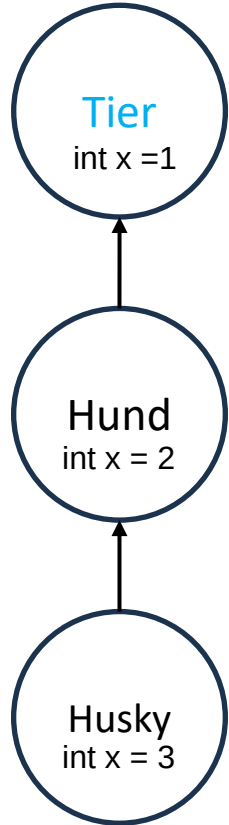


Regel: Attribute werden an Hand vom **statischen Typ** ausgewählt.

```
Tier t = new Tier();  
  
System.out.println(t.x);
```

Resultat: 1

Attribute:

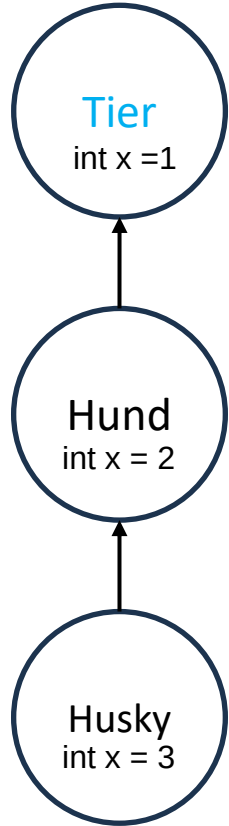


Regel: Attribute werden an Hand vom **statischen Typ** ausgewählt.

```
Tier t = new Hund();  
  
System.out.println(t.x);
```

Resultat: 1

Attribute:

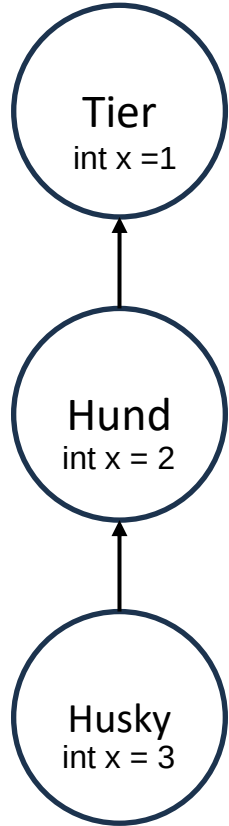


Regel: Attribute werden an Hand vom **statischen Typ** ausgewählt.

```
Tier t = new Husky();  
  
System.out.println(t.x);
```

Resultat: 1

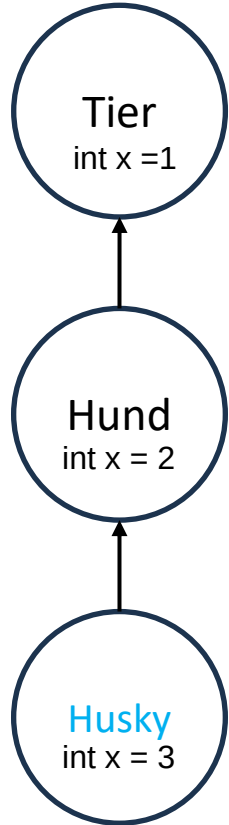
Attribute:



Regel: Attribute werden an Hand vom **statischen Typ** ausgewählt.

```
Husky t = new Husky();  
  
System.out.println(t.x);
```

Attribute:

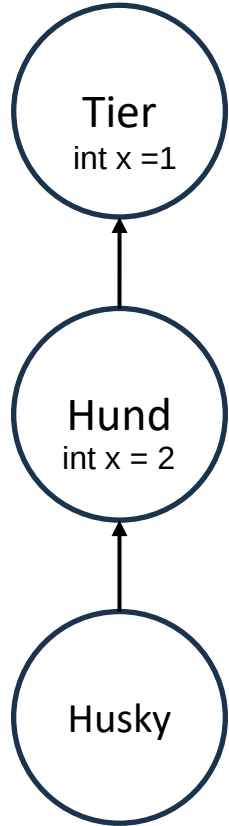


Regel: Attribute werden an Hand vom **statischen Typ** ausgewählt.

```
Husky t = new Husky();  
  
System.out.println(t.x);
```

Resultat: 3

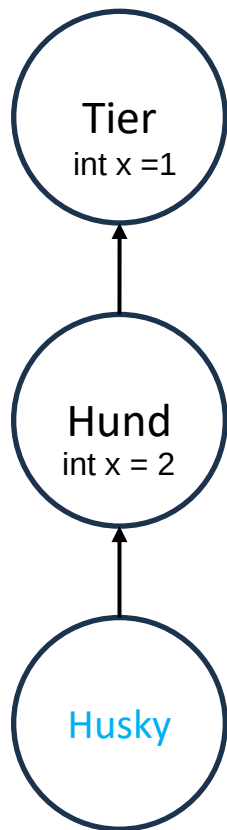
Attribute:



Regel: Attribute werden an Hand vom **statischen Typ** ausgewählt.

```
Husky t = new Husky();  
  
System.out.println(t.x);
```

Attribute:



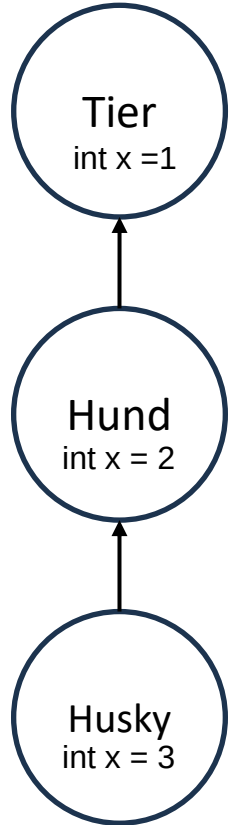
Regel: Attribute werden an Hand vom **statischen Typ** ausgewählt.

```
Husky t = new Husky();  
  
System.out.println(t.x);
```

Resultat: 2

Einer Klasse stehen grundsätzlich alle **nicht private** Variablen der Superklasse zur Verfügung. Wird die Variable explizit deklariert, wird die vererbte Variable “verdeckt” und ist nicht mehr zugänglich. → **Shadowing**

Attribute:

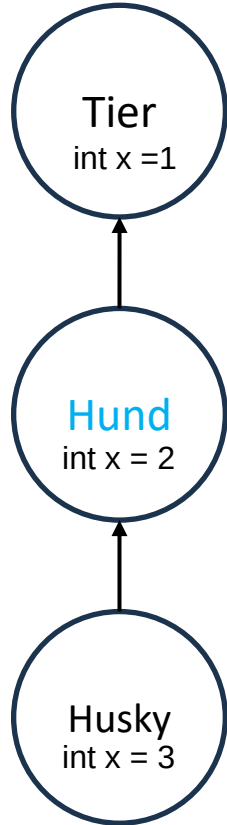


Regel: Attribute werden an Hand vom **statischen Typ** ausgewählt.

```
Husky t = new Husky();  
  
System.out.println(((Hund)t).x);
```

Resultat: 2

Attribute:



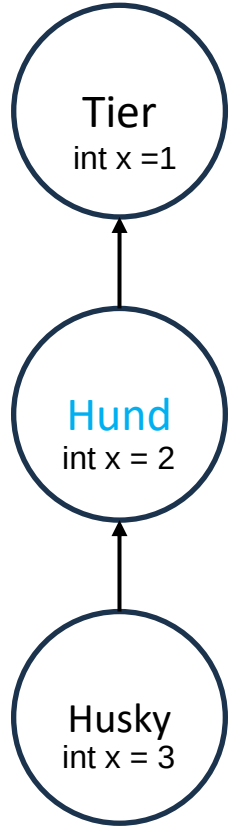
Regel: Attribute werden an Hand vom **statischen Typ** ausgewählt.

```
Husky t = new Husky();  
  
System.out.println(((Hund)t).x);
```

Resultat: 2

Durch den Cast sehen wir: Das neu definieren von **x** hat **keine** Auswirkung auf **x** der Superklasse.

Attribute:



```
Husky t = new Husky();
```

```
System.out.println(((Hund)t).x);
```



Hier wird implizit eine neue, temporäre Variable mit statischem Typ Hund erstellt.

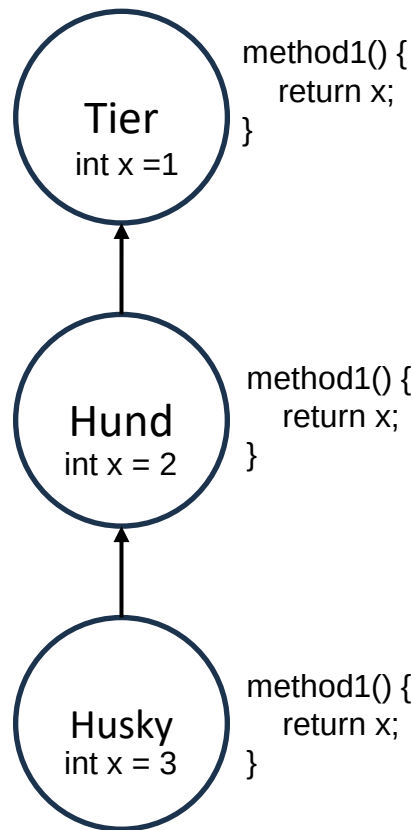
```
Husky t = new Husky();
```

```
Hund casted_t = (Hund) t;
```

```
System.out.println(casted_t.x);
```

Methodenwahl

Methoden:



Regel: Methoden* werden an Hand vom **dynamischen Typ** ausgewählt.

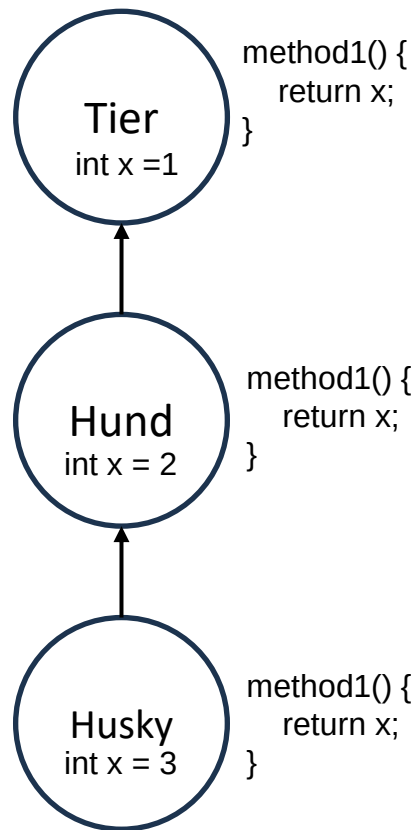
* ausser private, static und final Methoden (hier statischer Typ)

```
Tier t = new Husky();
```

```
System.out.println(t.method1());
```

Resultat: 3

Methoden:



Regel: Methoden* werden an Hand vom **dynamischen Typ** ausgewählt.

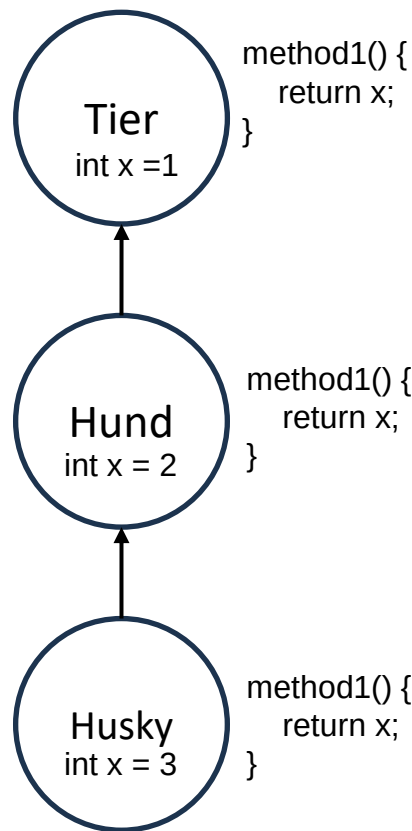
* ausser private, static und final Methoden (hier statischer Typ)

```
Hund t = new Husky();
```

```
System.out.println(t.method1());
```

Resultat: 3

Methoden:



Regel: Methoden* werden an Hand vom **dynamischen Typ** ausgewählt.

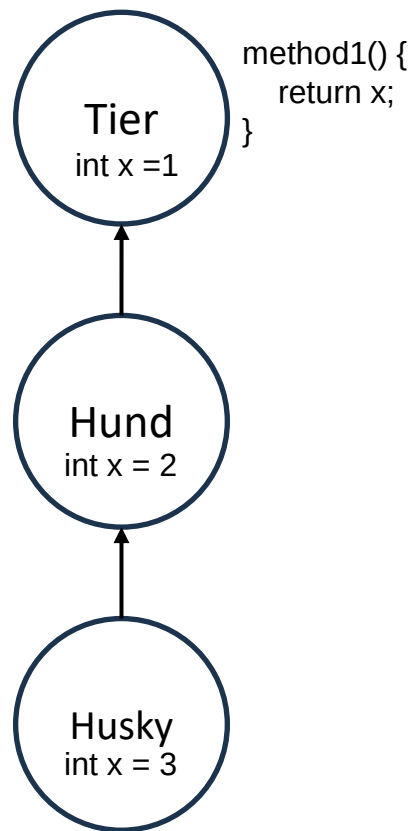
* ausser private, static und final Methoden (hier statischer Typ)

```
Hund t = new Hund();
```

```
System.out.println(t.method1());
```

Resultat: 2

Methoden:



Regel: Methoden* werden an Hand vom **dynamischen Typ** ausgewählt.

* ausser private, static und final Methoden (hier statischer Typ)

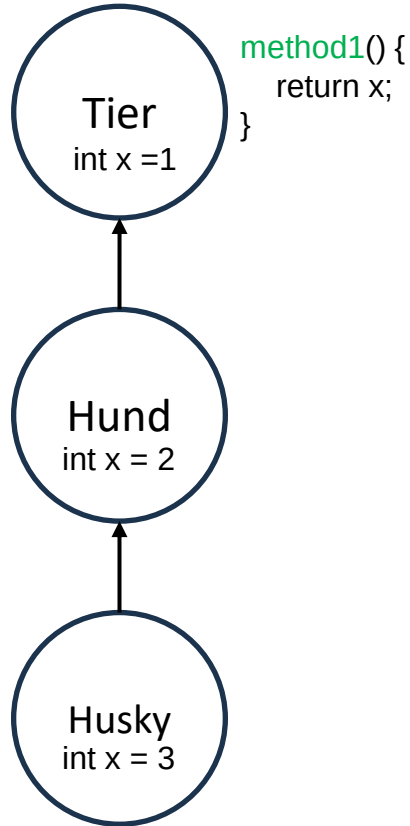
```
Tier t = new Husky();
```

```
System.out.println(t.method1());
```

Resultat: 1 ?

method1 existiert in der Husky Klasse nicht. Deshalb gehen wir durch alle Superklassen durch, bis wir eine solche Methode finden.

Methoden:



Regel: Methoden* werden an Hand vom **dynamischen Typ** ausgewählt.

* ausser private, static und final Methoden (hier statischer Typ)

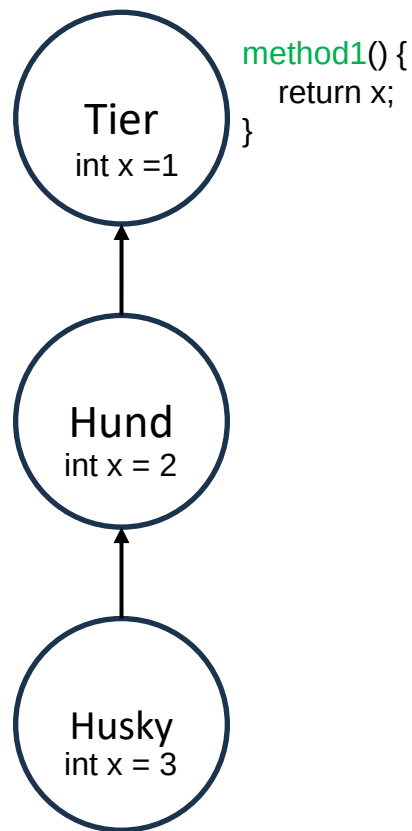
```
Tier t = new Husky();
```

```
System.out.println(t.method1());
```

Resultat: 1

method1 existiert in der Husky Klasse nicht. Deshalb gehen wir durch alle Superklassen durch, bis wir eine solche Methode finden.

Methoden:



Regel: Methoden* werden an Hand vom **dynamischen Typ** ausgewählt.

* ausser private, static und final Methoden (hier statischer Typ)

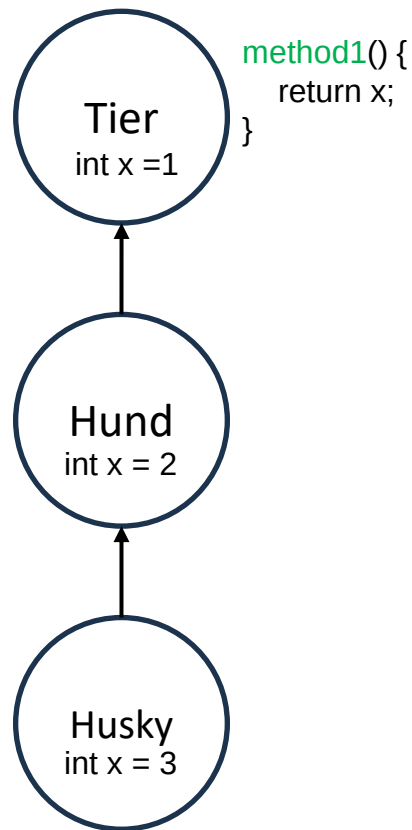
```
Tier t = new Husky();
```

```
System.out.println(t.method1());
```

Resultat: 1

method1 existiert in der Husky Klasse nicht. Deshalb gehen wir durch alle Superklassen durch, bis wir eine solche Methode finden.

Methoden:



Regel: Methoden* werden an Hand vom **dynamischen Typ** ausgewählt.

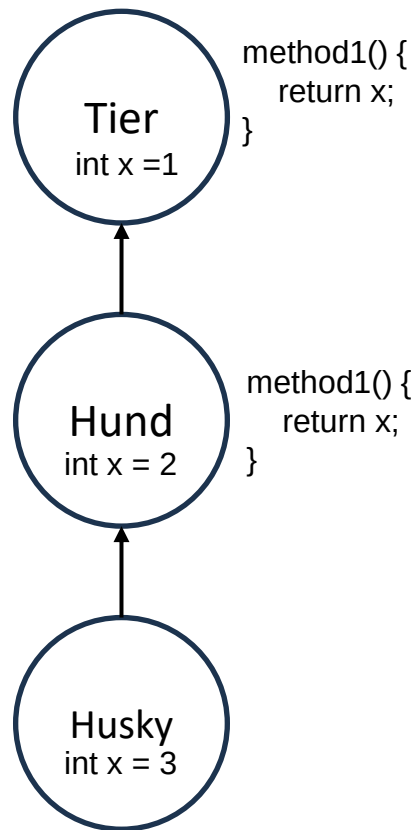
* ausser private, static und final Methoden (hier statischer Typ)

`method1` existiert in der **Husky** Klasse nicht. Deshalb gehen wir durch alle Superklassen durch, bis wir eine solche Methode finden.

Das Attribut `x` wird weiterhin statisch ausgewählt.

- Beim Kompilieren wird bestimmt, dass falls `method1` in der Klasse **Tier** aufgerufen wird, dass wir **immer** das Attribut `x` aus der Klasse **Tier** wählen.

Methoden:



Regel: Methoden* werden an Hand vom **dynamischen Typ** ausgewählt.

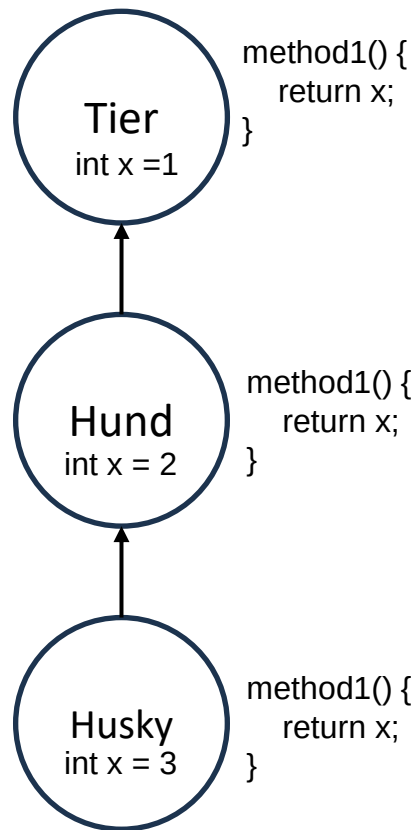
* ausser private, static und final Methoden (hier statischer Typ)

```
Tier t = new Husky();
```

```
System.out.println(t.method1());
```

Resultat: 2

Methoden:



Regel: Methoden* werden an Hand vom **dynamischen Typ** ausgewählt.

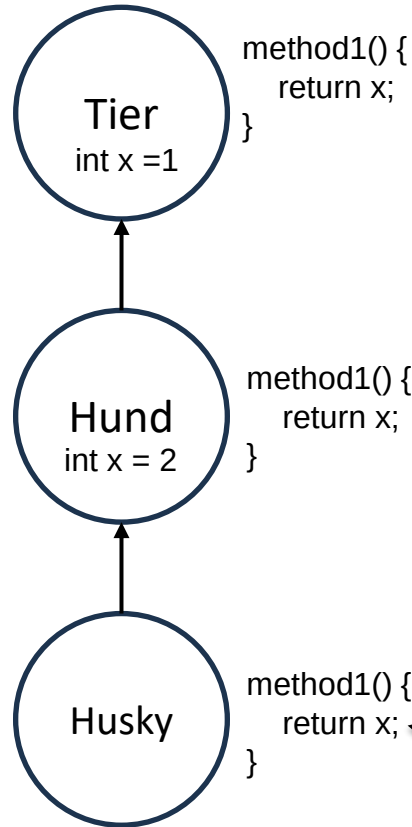
* ausser private, static und final Methoden (hier statischer Typ)

```
Tier t = new Husky();
```

```
System.out.println(t.method1());
```

Resultat: 3

Methoden:



Regel: Methoden* werden an Hand vom **dynamischen Typ** ausgewählt.

* ausser private, static und final Methoden (hier statischer Typ)

```
Tier t = new Husky();
```

```
System.out.println(t.method1());
```

Resultat: 2

Husky hat selbst kein Attribut x. Beim Kompilieren wird bestimmt, dass falls `method1` in der Klasse `Husky` aufgerufen wird, dass immer das Attribut `x` aus der Superklasse gewählt wird.

this und super

Keywords bei Objekten

- **new** MyClass(...)
 - Ruft einen Konstruktor von MyClass mit der entsprechenden Argumentenliste auf
- **super**(...)
 - Ruft einen Konstruktor der Superklasse mit der entsprechenden Argumentenliste auf

Keywords bei Objekten

- **this** ist eine Referenz auf das Objekt, aus dem wir gerade arbeiten und kann auch ganz normal als Objektreferenz behandelt werden
 - **this(...)**
 - Ruft den passenden Konstruktor auf
 - **this.field**
 - Gibt das angesprochene Objektattribut
 - **this.someMethod()**
 - Ruft die angesprochene Methode auf
 - **otherMethod(this, 7)**
 - Übergibt die Referenz auf das aktuelle Objekt

Wichtig!!

Attribute versus Methoden: Beispiel

```
class A {  
    String s = "in A";  
    String myS() { return s; }  
}
```

```
class B extends A {  
    String s = "in B";  
    @Override  
    String myS() { Return this.s; }  
}
```

```
B b = new B();  
System.out.println(b.s);           // in B  
System.out.println(b.myS());       // in B  
System.out.println((A) b.s);       // in A (static type!)  
System.out.println((A) b.myS());   // in B (dynamic type!)
```

Vererbung informell:

- 1) Statischer Typ entscheidet, welche Methoden zulässig sind. **(Subtyping)**
- 2) Dynamischer Typ entscheidet, welche Version der Methode verwendet wird. **(Dynamic binding)**
- 3) Welche Methode wird aufgerufen? **(overloading)**
 - > Die, die am besten passt!
 - > Entscheidet bereits bei 1)
- Bei Attributen: Statischer Typ entscheidet über beides. Auch Attribute werden vererbt!

Prüfungsaufgabe

- HS 21 (B)
- 10/40 Punkten
- Prüfungsdauer: 40 Minuten
- -> 10 Minuten Zeit

```

class Aaa {
    Integer s = 1;

    public void fct1() {
        System.out.println("Aaa " + s);
    }
}

class Mmm extends Aaa {
    String s = "Mmm";

    public void fct1() {
        System.out.println("Mmm fct1 " + s);
    }

    public void fct2() {
        System.out.println("Mmm fct2 " + s);
    }
}

class Nnn extends Aaa {
    String s = "Nnn";

    public void fct1() {
        System.out.println("Nnn fct1 " + s);
        fct3();
    }

    public void fct3() {
        System.out.println("Nnn fct3 " + s);
    }
}

```

```

class Bbb extends Nnn {
    String s = "Bbb";

    public void fct2() {
        System.out.println("Bbb fct2 " + s);
    }
}

class Ccc extends Nnn {
    String s = "Ccc";

    public void fct2() {
        super.fct1();
        System.out.println("Ccc fct2 " + s);
    }

    public void fct3() {
        System.out.println("Ccc fct3 " + s);
    }

    public String toString() {
        return s;
    }
}

class Ddd extends Ccc {
    String s = "Ddd";

    public void fct2() {
        System.out.println("Ddd fct2 " + s);
    }
}

```

Solutions

1. `Object ox = new Aaa();`
`Object oy = (Aaa) ox;`
`oy.fct1();`

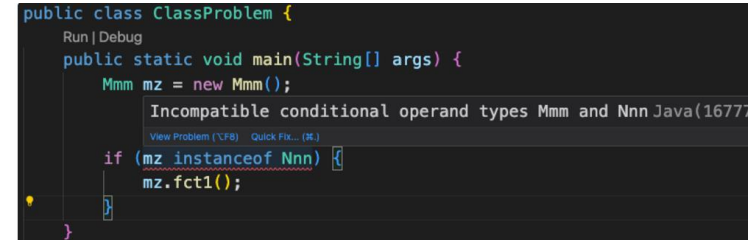
Error: fct1() is not defined in Object

2. `Aaa mx = new Mmm();`
`mx.fct1();`

Mmm fct1 Mmm

3. `Mmm mz = new Mmm();`
`if (mz instanceof Nnn) {`
`mz.fct1();`
`}`

Compile-Error



```
public class ClassProblem {  
    Run | Debug  
    public static void main(String[] args) {  
        Mmm mz = new Mmm();  
        if (mz instanceof Nnn) {  
            mz.fct1();  
        }  
    }  
}
```

Incompatible conditional operand types Mmm and Nnn Java(1677)

View Problem (F12) Quick Fix... (M...)

4. `Bbb bx = new Bbb();`
`bx.fct2();`

Bbb fct2 Bbb

Solutions

5. `Ccc cz = new Ddd();`
`cz.fct3();`

Ccc fct3 Ccc

6. `Ccc ca = new Ddd();`
`((Aaa) ca).fct1();`

Nnn fct1 Nnn
Ccc fct3 Ccc

7. `Ccc cx = new Ccc();`
`Ddd dy = (Ddd) cx;`
`dy.fct3();`

Exception (Runtime -> Downcast not possible)

8. `Ddd dx = new Ddd();`
`System.out.println(dx);`

Ccc

Vorbesprechung

Aufgabe 1: Loop- Invarianten

1. Um die Loop-Invariante einfacher schreiben zu können, dürfen Sie `min(arr, i)` benutzen. Hier steht `min(arr, i)` für das minimale Element in dem Array `arr` von Index 0 bis und mit Index `i`. Alternativ könnte man auch formale Notation benutzen, in dem man mit Quantoren arbeitet. Zum Beispiel, falls `m = min(arr, i)`, dann könnten Sie äquivalent folgendes schreiben

$$\forall 0 \leq j \leq i \ (arr[j] \leq m)$$

```
int min(int[] arr) {  
    // Precondition: arr != null 0 < arr.length  
    int m = arr[0];  
    int i = 1;  
  
    // Loop-Invariante:  
    while (i < arr.length) {  
        if (arr[i] < m) {  
            m = arr[i];  
        }  
  
        i++;  
    }  
  
    // Postcondition: m = min(arr, arr.length)  
    return m;  
}
```

Aufgabe 1: Loop- Invarianten

```
2. String append(String str1, String str2) {  
    // Precondition: str1 != null && str2 != null  
    String s1 = str1;  
    String s2 = str2;  
  
    // Loop-Invariante:  
    while (!s2.equals("")) {  
        s1 = s1 + s2.charAt(0);  
        s2 = s2.substring(1);  
    }  
  
    // Postcondition: s.equals(str1 + str2)  
    return s1;  
}
```

Achtung: Die Bedingung `str1 != null && str2 != null` ist wichtig, damit Aufrufe wie `s2.equals()`, `s2.charAt(0)` und `s2.substring(1)` überhaupt möglich sind. Der Aufruf `s2.substring(1)` produziert das gleiche Resultat wie `s2.substring(1, s2.length())`.

Aufgabe 2: Item Factory

In dieser Aufgabe implementieren Sie für eine Datenbank von Personengesundheitsdaten das Deklassifizieren von Einträgen (Task a) und das Verlinken von Einträgen (Task b). Alle Unteraufgaben können separat gelöst werden.

Die Datenbank selber ist bereits mit der Klasse `Database` implementiert. Die Datenbank hält eine Liste von Einträgen, welche durch die Klasse `Item` repräsentiert werden. Die folgenden 4 Paragraphen erklären alle in der Vorlage gegebenen Klassen im Detail.

Item Die Klasse `Item` repräsentiert einen Datenbankeintrag mit 4 Attributen: eine ID (int), ein Alter (int), einen Gesundheitswert (int), und ein Sicherheitslevel, welches durch die Klasse `Level` repräsentiert wird. Alter und Gesundheitswert sind immer ≥ 0 . Die Methoden `Item.getID()`, `Item.getAge()`, `Item.getHealth()`, `Item.getLevel()` geben jeweils die ID, das Alter, den Gesundheitswert, und das Sicherheitslevel eines Eintrags zurück. Die Methode `Item.setHealth(int newHealth)` setzt den Gesundheitswert auf `newHealth`. Die anderen Attribute können nicht geändert werden.

Level Die Klasse `Level` repräsentiert ein Sicherheitslevel. Ein Sicherheitslevel wird über eine Liste von Integern definiert, welches in einem Attribut der Klasse `Level` gespeichert wird und von der Methode `Level.getPoints()` zurückgegeben wird. Ein Level A ist *verwandt* mit einem Level B, falls die Summe der Werte in `A.getPoints()` gleich der Summe der Werte in `B.getPoints()` ist. Zum Beispiel ist das Level `[1,2,3,4]` verwandt mit den Levels `[10]` und `[4,6]` (die Summe ist überall 10), aber nicht mit dem Level `[4,5]`.

Aufgabe 2: Item Factory

ItemFactory Die Klasse `ItemFactory` wird verwendet, um Datenbankinträge zu erstellen. Die Methode `ItemFactory.createItem(Level level, int id, int age, int health)` gibt ein Exemplar der Klasse `Item` zurück, deren Attribute mit den Argumenten initialisiert wurden.

Database Die Klasse `Database` repräsentiert eine Datenbank und hat folgende vorgegebene Methoden:

- `Database.getItemFactory()` gibt ein Exemplar von `ItemFactory` zurück. Die `ItemFactory` `I` ist assoziiert mit der Datenbank `D`, falls `I` von `D.getItemFactory()` zurückgegeben wird.
 - `Database.add(Item item)` fügt der Datenbank den Eintrag `item` hinzu.
 - `Database.getItems()` gibt die Liste aller Einträge zurück, welcher der Datenbank hinzugefügt wurden. Sie dürfen annehmen, dass für eine Datenbank `D` alle Einträge in `D.getItems()` eine einzigartige ID haben, über `D.add` hinzugefügt wurden, über `D.getItemFactory()` erstellt wurden, und keiner anderen Datenbank hinzugefügt werden. Ein hinzugefügter Eintrag wird nie wieder entfernt.
1. Implementieren Sie die Methode `ItemFactory.createDeclass(Level level, int id, int targetId)`, die einen *Deklassifikationseintrag* zurückgibt. Ein Deklassifikationseintrag ist selber ein Eintrag, also ein Exemplar der Klasse `Item`. Ein Deklassifikationseintrag hat damit auch eine ID, ein Sicherheitslevel, ein Alter, und einen Gesundheitswert, welche von den

Aufgabe 2: Item Factory

entsprechenden getter-Methoden zurückgegeben werden. ID und Sicherheitslevel eines Deklassifikationseintrags sind jeweils das `id` und `level` Argument des `createDeclass` Aufrufs, mit welchem der Eintrag erstellt wurde. Das Alter und der Gesundheitswert eines Deklassifikationseintrags sind jeweils das `Alter` und der Gesundheitswert des *Zieleintrags* vom Deklassifikationseintrag. Der Zieleintrag von einem Deklassifikationseintrag `D` ist der Eintrag `E`, so dass

- `E.getID()` gleich dem Parameter `targetId` ist, mit welchem `D` erstellt wurde; *und*
- `E` aus der Datenbank ist, mit welcher die `ItemFactory` assoziiert ist, mit welcher `D` erstellt wurde.

Falls es keinen Zieleintrag gibt, wird eine `IllegalArgumentException` von der Methode `createDeclass` geworfen. Beachten Sie, dass Zieleinträge selber Deklassifikationseinträge sein können. Ein Aufruf der Methode `Item.setHealth(h)` auf einem Deklassifikationseintrag hat keinen Effekt; dies wird nicht in den Tests überprüft.

Ein Deklassifikationseintrag `R` *erreicht* einen Eintrag `A`, falls entweder `A` der Zieleintrag von `R` ist oder falls der Zieleintrag von `R` ein Deklassifikationseintrag ist, welcher `A` erreicht. Die Methode `createDeclass` wirft eine `IllegalArgumentException`, falls der zurückzugebene Deklassifikationseintrag `R` einen Eintrag erreicht, dessen Level verwandt ist mit dem Level von `R`. Zur Erinnerung: Der Paragraph über die Klasse `Level` erklärt, wann zwei Level verwandt sind.

Aufgabe 2: Item Factory

entsprechenden getter-Methoden zurückgegeben werden. ID und Sicherheitslevel eines Deklassifikationseintrags sind jeweils das `id` und `level` Argument des `createDeclass` Aufrufs, mit welchem der Eintrag erstellt wurde. Das Alter und der Gesundheitswert eines Deklassifikationseintrags sind jeweils das `Alter` und der Gesundheitswert des *Zieleintrags* vom Deklassifikationseintrag. Der Zieleintrag von einem Deklassifikationseintrag `D` ist der Eintrag `E`, so dass

- `E.getID()` gleich dem Parameter `targetId` ist, mit welchem `D` erstellt wurde; *und*
- `E` aus der Datenbank ist, mit welcher die `ItemFactory` assoziiert ist, mit welcher `D` erstellt wurde.

Falls es keinen Zieleintrag gibt, wird eine `IllegalArgumentException` von der Methode `createDeclass` geworfen. Beachten Sie, dass Zieleinträge selber Deklassifikationseinträge sein können. Ein Aufruf der Methode `Item.setHealth(h)` auf einem Deklassifikationseintrag hat keinen Effekt; dies wird nicht in den Tests überprüft.

Ein Deklassifikationseintrag `R` *erreicht* einen Eintrag `A`, falls entweder `A` der Zieleintrag von `R` ist oder falls der Zieleintrag von `R` ein Deklassifikationseintrag ist, welcher `A` erreicht. Die Methode `createDeclass` wirft eine `IllegalArgumentException`, falls der zurückzugebene Deklassifikationseintrag `R` einen Eintrag erreicht, dessen Level verwandt ist mit dem Level von `R`. Zur Erinnerung: Der Paragraph über die Klasse `Level` erklärt, wann zwei Level verwandt sind.

Aufgabe 2: Item Factory

2. Implementieren Sie die Methode `Database.createLink(List<Integer> ids)`. Der Methodenaufruf `D.createLink(ids)` *verlinkt* alle Einträge der Datenbank `D` miteinander, welche eine ID haben, die im Argument `ids` enthalten ist. Wenn `E.setHealth(h)` auf einem Eintrag `E` aufgerufen wird, dann wird der Gesundheitswert aller Einträge, welche mit `E` verlinkt sind, auf das Argument `h` gesetzt. Einträge können beliebig oft verlinkt werden und verlinken ist transitiv, das heisst, wenn ein Eintrag `A` mit einem Eintrag `B` verlinkt ist und `B` mit einem Eintrag `C` verlinkt ist, dann ist `A` auch mit `C` verlinkt. Verlinken ist auch immer symmetrisch, das heisst, wenn `A` mit `B` verlinkt ist, dann ist auch `B` mit `A` verlinkt. Zusätzlich ist verlinken reflexiv, das heisst, ein Eintrag ist immer mit sich selber verlinkt.

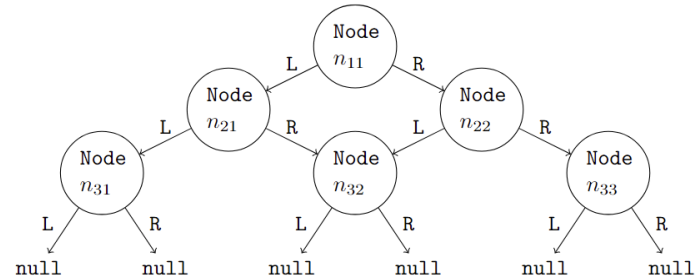
Der Aufruf `D.createLink(ids)` soll eine `IllegalArgumentException` werfen, falls es eine ID im Argument `ids` gibt, für welche es keinen Eintrag mit der gleichen ID in der Datenbank `D` gibt.

Wir geben zwei Testdateien zur Verfügung. “DatabaseTest.java” enthält Tests, welche wir an einer Prüfung geben würden. “GradingDatabaseTest.java” enthält Tests, welche wir zum Korrigieren einer Prüfung verwenden würden. Testen Sie Ihre Lösung zuerst ausgiebig mit “DatabaseTest.java” (am besten fügen Sie selber neue Tests hinzu) und dann können Sie “GradingDatabaseTest.java” verwenden, um zu sehen wie Ihre Lösung an einer Prüfung abgeschnitten hätte.

Aufgabe 3: Pyramide

Die Klasse `Node` repräsentiert einen Knoten in einem gerichteten Graphen, wobei es für jeden Knoten n_1 höchstens zwei gerichtete Kanten von n_1 zu anderen Knoten n_2, n_3 geben kann (n_2 und n_3 können gleich sein). Wir unterscheiden dabei zwischen dem linken und dem rechten Knoten. Die Methode `Node.getLeft()` gibt den linken Knoten und `Node.getRight()` den rechten Knoten zurück (als `Node`-Objekt). Wenn der linke Knoten von n_1 nicht existiert, dann gibt `Node.getLeft()` `null` zurück (analog für den rechten Knoten).

Das Ziel dieser Aufgabe ist, für ein `Node`-Objekt zu entscheiden, ob der durch das `Node`-Objekt definierte Graph einer Pyramide entspricht. Zum Beispiel entspricht der folgende Graph einer Pyramide.



Aufgabe 3: Pyramide

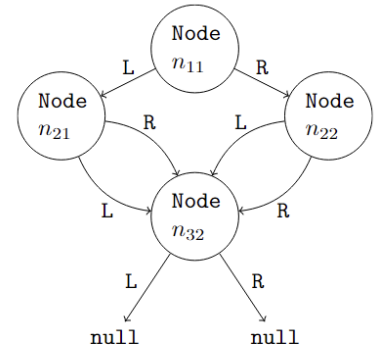
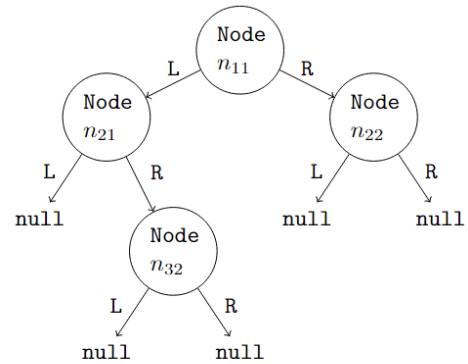
Beachten Sie, dass der rechte Knoten von n_{21} gleich ist wie der linke Knoten von n_{22} (das heisst die Node-Objekte sind gleich!). Ein Graph (wie oben repräsentiert) definiert eine Pyramide genau dann, wenn folgende Bedingungen gelten:

- Der Graph kann in $k \geq 1$ Stufen (Stufe 1, Stufe 2,..., Stufe k) aufgeteilt werden, wobei Stufe i aus i unterschiedlichen Knoten $n_{i1}, n_{i2}, \dots, n_{ii}$ besteht. Falls der Graph k Stufen hat, dann hat dieser genau $\frac{k(k+1)}{2}$ unterschiedliche Knoten (Knoten aus verschiedenen Stufen sind unterschiedlich).
- Für Stufe i ($1 \leq i < k$) gilt: der linke Knoten von n_{ij} ($1 \leq j \leq i$) ist durch $n_{(i+1)j}$ gegeben und der rechte Knoten von n_{ij} ist durch $n_{(i+1)(j+1)}$ gegeben.
- Für Stufe k gilt: es gibt keinen linken und keinen rechten Knoten für n_{kj} ($1 \leq j \leq k$).

Die folgenden Graphen entsprechen zum Beispiel keinen Pyramiden:
Implementieren Sie die `boolean isPyramid(Node node)`-Methode, welche, für den Graph G durch `node` definiert, entscheidet, ob G eine Pyramide definiert. Sie dürfen annehmen, dass G keine Zyklen hat. Die Methode soll eine `IllegalArgumentException` werfen, wenn das Argument `null` ist.

Tipp: Prüfen Sie die Bedingungen Stufe für Stufe, beginnend bei Stufe 1.

Aufgabe 3: Pyramide



Aufgabe 4: Rechnungen (erweitert)

In dieser Aufgabe erweitern Sie eine vorherige Aufgabe, in welcher ein System für Stromverbräuche Rechnungen erstellt. Konkret gibt es drei Erweiterungen: (1) Es sollen auch nicht korrekt formatierte Eingabedateien gehandhabt werden. (2) Ein Kunde kann eine beliebige Anzahl von Verbrauchswerten haben. (3) Es gibt eine neue Unteraufgabe b. In der folgenden Aufgabenbeschreibung für Unteraufgabe a sind die Änderung in **bold** markiert.

- a) Vervollständigen Sie die `process`-Methode in der Klasse `Bills`. Die Methode hat zwei Argumente: einen `Scanner`, von dem Sie den Inhalt der Eingabedatei lesen sollen, und einen `PrintStream`, in welchen Sie die unten beschriebenen Informationen schreiben.

Ihr Programm muss **auch mit manchen nicht korrekt formatierten Eingabedateien umgehen. Die Aufgabestellung gibt an, wie mit nicht korrekt formatierten Eingaben umzugehen ist.** Ein Beispiel einer korrekt formatierten Datei finden Sie im Projekt unter dem Namen "Data.txt". Exceptions im Zusammenhang mit Ein- und Ausgabe können Sie ignorieren.

Eine valide Eingabedatei enthält Zeilen, die entweder den Tarif, der angewendet werden soll, oder die Daten für den Stromverbrauch eines Kunden beschreiben. Der Verbrauch eines Kunden ist niemals grösser als 100000 Kilowattstunden.

Eine Tarifbeschreibung hat folgendes Format:

$$\text{Tarif_}n_l_1_p_1 \dots l_n_p_n$$

Loop-Invariante

```
public static int compute(int i) {  
    // Precondition: i >= 0  
  
    int n;  
    int k;  
  
    n = 0;  
    k = i;  
  
    //Loop-Invariante:  
  
    While (k != 0) {  
        n += k % 10;  
        k = k / 10;  
    }  
  
    //Postcondition: quersumme(i) == n  
  
    return n;  
}  
//quersumme(i) gibt die Quersumme von int i zurück (i >= 0)
```

Loop-Invariante vorgehen:

- Code anschauen: was passiert?

```
public static int compute(int i) {  
    // Precondition: i >= 0  
  
    int n;  
    int k;  
    n = 0;  
    k = i;  
  
    //Loop-Invariante:  
    While (k != 0) {  
        n += k % 10;  
        k = k / 10;  
    }  
  
    //Postcondition: quersumme(i) == n  
    return n;  
} //quersumme(i) gibt die Quersumme von int i zurück (i >= 0)
```

Loop-Invariante vorgehen:

- Code anschauen: was passiert?
- Initialisierung

```
public static int compute(int i) {  
    // Precondition: i >= 0  
  
    int n;  
    int k;  
  
    n = 0;  
    k = i;  
  
    //Loop-Invariante:  
    While (k != 0) {  
        n += k % 10;  
        k = k / 10;  
    }  
  
    //Postcondition: quersumme(i) == n  
  
    return n;  
}  
//quersumme(i) gibt die Quersumme von int i zurück (i >= 0)
```



Loop-Invariante vorgehen:

- Code anschauen: was passiert?
 - Initialisierung
 - Bis $k == 0$ ist, wird k in jeder Iteration durch 10 dividiert

```
public static int compute(int i) {
```

```
    // Precondition: i >= 0
```

```
    int n;
```

```
    int k;
```

```
    n = 0;
```

```
    k = i;
```

Bsp:

k = 1234

k / 10 = 123

```
    //Loop-Invariante:
```

```
    While (k != 0) {
```

```
        n += k % 10;
```

```
        k = k / 10;
```

```
    }
```

```
    //Postcondition: quersumme(i) == n
```

```
    return n;
```

```
} //quersumme(i) gibt die Quersumme von int i zurück (i >= 0)
```

Loop-Invariante vorgehen:

- Code anschauen: was passiert?
 - Initialisierung
 - Bis k == 0 ist, wird k in jeder Iteration durch 10 dividiert

```
public static int compute(int i) {
```

```
    // Precondition: i >= 0
```

```
    int n;
```

```
    int k;
```

```
    n = 0;
```

```
    k = i;
```

Bsp:

k = 1234

k / 10 = 123

```
    //Loop-Invariante:
```

```
    While (k != 0) {
```

```
        n += k % 10;
```

```
        k = k / 10;
```

```
    }
```

```
    //Postcondition: quersumme(i) == n
```

```
    return n;
```

```
} //quersumme(i) gibt die Quersumme von int i zurück (i >= 0)
```

Loop-Invariante vorgehen:

- Code anschauen: was passiert?
 - Initialisierung
 - Bis k == 0 ist, wird k in jeder Iteration durch 10 dividiert → **immer die letzte Ziffer wird "abgeschnitten"**

```
public static int compute(int i) {  
    // Precondition: i >= 0  
  
    int n;  
    int k;  
  
    n = 0;  
    k = i;  
  
    //Loop-Invariante:  
    While (k != 0) {  
        n += k % 10;  
        k = k / 10;  
    }  
  
    //Postcondition: quersumme(i) == n  
  
    return n;  
}  
//quersumme(i) gibt die Quersumme von int i zurück (i >= 0)
```

Loop-Invariante vorgehen:

- Code anschauen: was passiert?
 - Initialisierung
 - Bis $k == 0$ ist, wird k in jeder Iteration durch 10 dividiert → **immer die letzte Ziffer wird "abgeschnitten"**
 - In jeder Iteration wird $k \% 10$ zu n addiert

```
public static int compute(int i) {
```

```
    // Precondition: i >= 0
```

```
    int n;
```

```
    int k;
```

```
    n = 0;
```

```
    k = i;
```

Bsp:

k = 1234

k % 10 = 4

```
    //Loop-Invariante:
```

```
    While (k != 0) {
```

```
        n += k % 10;
```

```
        k = k / 10;
```

```
    }
```

```
    //Postcondition: quersumme(i) == n
```

```
    return n;
```

```
} //quersumme(i) gibt die Quersumme von int i zurück (i >= 0)
```

Loop-Invariante vorgehen:

- Code anschauen: was passiert?
 - Initialisierung
 - Bis $k == 0$ ist, wird k in jeder Iteration durch 10 dividiert → **immer die letzte Ziffer wird "abgeschnitten"**
 - In jeder Iteration wird $k \% 10$ zu n addiert

```
public static int compute(int i) {
```

```
    // Precondition: i >= 0
```

```
    int n;
```

```
    int k;
```

```
    n = 0;
```

```
    k = i;
```

Bsp:

k = 1234

k % 10 = 4

```
    //Loop-Invariante:
```

```
    While (k != 0) {
```

```
        n += k % 10;
```

```
        k = k / 10;
```

```
    }
```

```
    //Postcondition: quersumme(i) == n
```

```
    return n;
```

```
} //quersumme(i) gibt die Quersumme von int i zurück (i >= 0)
```

Loop-Invariante vorgehen:

- Code anschauen: was passiert?
 - Initialisierung
 - Bis $k == 0$ ist, wird k in jeder Iteration durch 10 dividiert → **immer die letzte Ziffer wird "abgeschnitten"**
 - In jeder Iteration wird $k \% 10$ zu n addiert → **immer die letzte Ziffer wird dazu addiert**

```

public static int compute(int i) {
    // Precondition: i >= 0

    int n;

    int k;

    n = 0;

    k = i;

    //Loop-Invariante:

    While (k != 0) {
        n += k % 10;

        k = k / 10;
    }

    //Postcondition: quersumme(i) == n

    return n;
} //quersumme(i) gibt die Quersumme von int i zurück (i >= 0)

```

Loop-Invariante vorgehen:

- Code anschauen: was passiert?
 - Initialisierung
 - Bis $k == 0$ ist, wird k in jeder Iteration durch 10 dividiert → immer die letzte Ziffer wird “abgeschnitten”
 - In jeder Iteration wird $k \% 10$ zu n addiert → immer die letzte Ziffer wird dazu addiert
 - **=> $n ==$ alle Ziffern von i zusammen addiert**

```

public static int compute(int i) {
    // Precondition: i >= 0

    int n;

    int k;

    n = 0;

    k = i;

    //Loop-Invariante:

    While (k != 0) {
        n += k % 10;

        k = k / 10;
    }

    //Postcondition: quersumme(i) == n

    return n;
} //quersumme(i) gibt die Quersumme von int i zurück (i >= 0)

```

Loop-Invariante vorgehen:

- Code anschauen: was passiert?
 - Initialisierung
 - Bis $k == 0$ ist, wird k in jeder Iteration durch 10 dividiert → immer die letzte Ziffer wird “abgeschnitten”
 - In jeder Iteration wird $k \% 10$ zu n addiert → immer die letzte Ziffer wird dazu addiert
 - **=> $n ==$ alle Ziffern von i zusammen addiert.**
 - das ist die **Quersumme** von i

```
public static int compute(int i) {  
    // Precondition: i >= 0  
  
    int n;  
    int k;  
  
    n = 0;  
    k = i;  
  
    //Loop-Invariante:  
    While (k != 0) {  
        n += k % 10;  
        k = k / 10;  
    }  
  
    //Postcondition: quersumme(i) == n  
  
    return n;  
}  
//quersumme(i) gibt die Quersumme von int i zurück (i >= 0)
```



Loop-Invariante vorgehen:

- Code anschauen: was passiert?
 - Initialisierung
 - Bis $k == 0$ ist, wird k in jeder Iteration durch 10 dividiert → immer die letzte Ziffer wird “abgeschnitten”
 - In jeder Iteration wird $k \% 10$ zu n addiert → immer die letzte Ziffer wird dazu addiert
 - **=> $n ==$ alle Ziffern von i zusammen addiert.**
 - das ist die **Quersumme** von I
- Postcondition anschauen

```

public static int compute(int i) {
    // Precondition: i >= 0

    int n;

    int k;

    n = 0;
    k = i;

    //Loop-Invariante:
    While (k != 0) {
        n += k % 10;
        k = k / 10;
    }

    //Postcondition: quersumme(i) == n

    return n;
} //quersumme(i) gibt die Quersumme von int i zurück (i >= 0)

```



Loop-Invariante vorgehen:

- Code anschauen: was passiert?
 - Initialisierung
 - Bis $k == 0$ ist, wird k in jeder Iteration durch 10 dividiert → immer die letzte Ziffer wird “abgeschnitten”
 - In jeder Iteration wird $k \% 10$ zu n addiert → immer die letzte Ziffer wird dazu addiert
 - **=> $n ==$ alle Ziffern von i zusammen addiert.**
 - das ist die **Quersumme** von i
- Postcondition anschauen
 - Unsere Vermutung war also richtig: die Quersumme von i wird berechnet

```

public static int compute(int i) {
    // Precondition: i >= 0

    int n;

    int k;

    n = 0;

    k = i;

    //Loop-Invariante:

    While (k != 0) {
        n += k % 10;

        k = k / 10;
    }

    //Postcondition: quersumme(i) == n

    return n;
} //quersumme(i) gibt die Quersumme von int i zurück (i >= 0)

```

Loop-Invariante vorgehen (Fortsetzung):

- Am Schluss wollen wir auf die Postcondition kommen. Aber $\text{quersumme}(i) == n$ stimmt am Anfang im Allgemeinen definitiv nicht
- $\text{quersumme}(i) != 0$

```

public static int compute(int i) {
    // Precondition: i >= 0

    int n;

    int k;

    n = 0;

    k = i;

    //Loop-Invariante:

    While (k != 0) {
        n += k % 10;

        k = k / 10;
    }

    //Postcondition: quersumme(i) == n

    return n;
} //quersumme(i) gibt die Quersumme von int i zurück (i >= 0)

```

Loop-Invariante vorgehen (Fortsetzung):

- Am Schluss wollen wir auf die Postcondition kommen. Aber $\text{quersumme}(i) == n$ stimmt am Anfang im Allgemeinen definitiv nicht
- $\text{quersumme}(i) != 0$
- **k** und **n** werden während dem Loop verändert, also müssen diese irgendwie in der Invariante vorkommen

```

public static int compute(int i) {
    // Precondition: i >= 0

    int n;

    int k;

    n = 0;

    k = i;

    //Loop-Invariante:

    While (k != 0) {
        n += k % 10;

        k = k / 10;
    }

    //Postcondition: quersumme(i) == n

    return n;
} //quersumme(i) gibt die Quersumme von int i zurück (i >= 0)

```

Loop-Invariante vorgehen (Fortsetzung):

- Am Schluss wollen wir auf die Postcondition kommen. Aber $\text{quersumme}(i) == n$ stimmt am Anfang im Allgemeinen definitiv nicht
- $\text{quersumme}(i) != 0$
- **k** und **n** werden während dem Loop verändert, also müssen diese irgendwie in der Invariante vorkommen
- Versuch 1: $\text{quersumme}(k) == n$

```

public static int compute(int i) {
    // Precondition: i >= 0

    int n;

    int k;

    n = 0;

    k = i;

    //Loop-Invariante:

    While (k != 0) {
        n += k % 10;

        k = k / 10;
    }

    //Postcondition: quersumme(i) == n

    return n;
} //quersumme(i) gibt die Quersumme von int i zurück (i >= 0)

```

Loop-Invariante vorgehen (Fortsetzung):

- Am Schluss wollen wir auf die Postcondition kommen. Aber $\text{quersumme}(i) == n$ stimmt am Anfang im Allgemeinen definitiv nicht
- $\text{quersumme}(i) != 0$
- **k** und **n** werden während dem Loop verändert, also müssen diese irgendwie in der Invariante vorkommen
- Versuch 1: $\text{quersumme}(k) == n$
- Prüfen:
 - Anfangs: $\text{quersumme}(i) == 0$ 

```

public static int compute(int i) {
    // Precondition: i >= 0

    int n;

    int k;

    n = 0;

    k = i;

    //Loop-Invariante:

    While (k != 0) {
        n += k % 10;

        k = k / 10;
    }

    //Postcondition: quersumme(i) == n

    return n;
} //quersumme(i) gibt die Quersumme von int i zurück (i >= 0)

```

Loop-Invariante vorgehen (Fortsetzung):

- Am Schluss wollen wir auf die Postcondition kommen. Aber $\text{quersumme}(i) == n$ stimmt am Anfang im Allgemeinen definitiv nicht
- $\text{quersumme}(i) != 0$
- **k** und **n** werden während dem Loop verändert, also müssen diese irgendwie in der Invariante vorkommen
- Versuch 1: $\text{quersumme}(k) == n$
- Prüfen:
 - Anfangs: $\text{quersumme}(i) == 0$
 - Wir haben links also $\text{quersumme}(i)$ “zu viel”



```

public static int compute(int i) {
    // Precondition: i >= 0

    int n;

    int k;

    n = 0;

    k = i;

    //Loop-Invariante:

    While (k != 0) {
        n += k % 10;

        k = k / 10;
    }

    //Postcondition: quersumme(i) == n

    return n;
} //quersumme(i) gibt die Quersumme von int i zurück (i >= 0)

```

Loop-Invariante vorgehen (Fortsetzung):

- Am Schluss wollen wir auf die Postcondition kommen. Aber $\text{quersumme}(i) == n$ stimmt am Anfang im Allgemeinen definitiv nicht
- $\text{quersumme}(i) != 0$
- **k** und **n** werden während dem Loop verändert, also müssen diese irgendwie in der Invariante vorkommen
- Versuch 1: $\text{quersumme}(k) == n$
- Prüfen:
 - Anfangs: $\text{quersumme}(i) == 0$ 
 - Wir haben links also $\text{quersumme}(i)$ “zu viel”
- Versuch 2: $\text{quersumme}(k) - \text{quersumme}(i) == n$

```

public static int compute(int i) {
    // Precondition: i >= 0

    int n;

    int k;

    n = 0;

    k = i;

    //Loop-Invariante:

    While (k != 0) {
        n += k % 10;

        k = k / 10;
    }

    //Postcondition: quersumme(i) == n

    return n;
} //quersumme(i) gibt die Quersumme von int i zurück (i >= 0)

```

Loop-Invariante vorgehen (Fortsetzung):

- Am Schluss wollen wir auf die Postcondition kommen. Aber $\text{quersumme}(i) == n$ stimmt am Anfang im Allgemeinen definitiv nicht
- $\text{quersumme}(i) != 0$
- **k** und **n** werden während dem Loop verändert, also müssen diese irgendwie in der Invariante vorkommen
- Versuch 1: $\text{quersumme}(k) == n$
- Prüfen:
 - Anfangs: $\text{quersumme}(i) == 0$ 
 - Wir haben links also $\text{quersumme}(i)$ “zu viel”
- Versuch 2: $\text{quersumme}(k) - \text{quersumme}(i) == n$
- Prüfen:
 - Anfangs: $\text{quersumme}(i) - \text{quersumme}(i) == 0$

```

public static int compute(int i) {
    // Precondition: i >= 0

    int n;

    int k;

    n = 0;

    k = i;

    //Loop-Invariante:

    While (k != 0) {
        n += k % 10;

        k = k / 10;
    }

    //Postcondition: quersumme(i) == n

    return n;
} //quersumme(i) gibt die Quersumme von int i zurück (i >= 0)

```

Loop-Invariante vorgehen (Fortsetzung):

- Am Schluss wollen wir auf die Postcondition kommen. Aber $\text{quersumme}(i) == n$ stimmt am Anfang im Allgemeinen definitiv nicht
- $\text{quersumme}(i) != 0$
- **k** und **n** werden während dem Loop verändert, also müssen diese irgendwie in der Invariante vorkommen
- Versuch 1: $\text{quersumme}(k) == n$
- Prüfen:
 - Anfangs: $\text{quersumme}(i) == 0$ 
 - Wir haben links also $\text{quersumme}(i)$ “zu viel”
- Versuch 2: $\text{quersumme}(k) - \text{quersumme}(i) == n$
- Prüfen:
 - Anfangs: $\text{quersumme}(i) - \text{quersumme}(i) == 0$
 - Ende: $\text{quersumme}(0) - \text{quersumme}(i) == \text{quersumme}(i)$ 

```

public static int compute(int i) {
    // Precondition: i >= 0

    int n;

    int k;

    n = 0;

    k = i;

    //Loop-Invariante:

    While (k != 0) {
        n += k % 10;

        k = k / 10;
    }

    //Postcondition: quersumme(i) == n

    return n;
} //quersumme(i) gibt die Quersumme von int i zurück (i >= 0)

```

Loop-Invariante vorgehen (Fortsetzung):

- Am Schluss wollen wir auf die Postcondition kommen. Aber $\text{quersumme}(i) == n$ stimmt am Anfang im Allgemeinen definitiv nicht
- $\text{quersumme}(i) != 0$
- **k** und **n** werden während dem Loop verändert, also müssen diese irgendwie in der Invariante vorkommen
- Versuch 1: $\text{quersumme}(k) == n$
- Prüfen:
 - Anfangs: $\text{quersumme}(i) == 0$ 
 - Wir haben links also $\text{quersumme}(i)$ “zu viel”
- Versuch 2: $\text{quersumme}(k) - \text{quersumme}(i) == n$
- Prüfen:
 - Anfangs: $\text{quersumme}(i) - \text{quersumme}(i) == 0$
 - Ende: $\text{quersumme}(0) - \text{quersumme}(i) == \text{quersumme}(i)$ 
 - Das Vorzeichen stimmt nicht

```

public static int compute(int i) {
    // Precondition: i >= 0

    int n;

    int k;

    n = 0;

    k = i;

    //Loop-Invariante:

    While (k != 0) {
        n += k % 10;

        k = k / 10;
    }

    //Postcondition: quersumme(i) == n

    return n;
} //quersumme(i) gibt die Quersumme von int i zurück (i >= 0)

```

Loop-Invariante vorgehen (Fortsetzung):

- Versuch 3: $\text{quersumme}(i) - \text{quersumme}(k) == n$

```

public static int compute(int i) {
    // Precondition: i >= 0

    int n;

    int k;

    n = 0;

    k = i;

    //Loop-Invariante:

    While (k != 0) {
        n += k % 10;

        k = k / 10;
    }

    //Postcondition: quersumme(i) == n

    return n;
} //quersumme(i) gibt die Quersumme von int i zurück (i >= 0)

```

Loop-Invariante vorgehen (Fortsetzung):

- Versuch 3: $\text{quersumme}(i) - \text{quersumme}(k) == n$
- Prüfen:
 - Anfangs: $\text{quersumme}(i) - \text{quersumme}(i) == 0$

```

public static int compute(int i) {
    // Precondition: i >= 0

    int n;

    int k;

    n = 0;

    k = i;

    //Loop-Invariante:

    While (k != 0) {
        n += k % 10;

        k = k / 10;
    }

    //Postcondition: quersumme(i) == n

    return n;
} //quersumme(i) gibt die Quersumme von int i zurück (i >= 0)

```

Loop-Invariante vorgehen (Fortsetzung):

- Versuch 3: $\text{quersumme}(i) - \text{quersumme}(k) == n$
- Prüfen:
 - Anfangs: $\text{quersumme}(i) - \text{quersumme}(i) == 0$
 - Nach jeder Iteration:
 - $\text{quersumme}(i) - \text{quersumme}(k/10) == n + k\%10$
 - "k ohne letzte Ziffer" "letzte Ziffer von k"

```

public static int compute(int i) {
    // Precondition: i >= 0

    int n;

    int k;

    n = 0;

    k = i;

    //Loop-Invariante:

    While (k != 0) {
        n += k % 10;

        k = k / 10;
    }

    //Postcondition: quersumme(i) == n

    return n;
} //quersumme(i) gibt die Quersumme von int i zurück (i >= 0)

```

Loop-Invariante vorgehen (Fortsetzung):

- Versuch 3: $\text{quersumme}(i) - \text{quersumme}(k) == n$
- Prüfen:
 - Anfangs: $\text{quersumme}(i) - \text{quersumme}(i) == 0$
 - Nach jeder Iteration:
 - $\text{quersumme}(i) - \text{quersumme}(k/10) == n + k\%10$
 - "k ohne letzte Ziffer" "letzte Ziffer von k"
 - Ende: $\text{quersumme}(i) - \text{quersumme}(0) == \text{quersumme}(i)$
- Korrekt !!

```
public static int compute(int i) {
```

```
    // Precondition: i >= 0
```

```
    int n;
```

```
    int k;
```

```
    n = 0;
```

```
    k = i;
```

```
    //Loop-Invariante:
```

```
    While (k != 0) {
```

```
        n += k % 10;
```

```
        k = k / 10;
```

```
    }
```

```
    //Postcondition: quersumme(i) == n
```

```
    return n;
```

```
} //quersumme(i) gibt die Quersumme von int i zurück (i >= 0)
```

Loop-Invariante vorgehen (Fortsetzung):

Inv: $\text{quersumme}(i) - \text{quersumme}(k) == n$

- Es fehlt noch etwas!



```
public static int compute(int i) {  
    // Precondition: i >= 0  
  
    int n;  
    int k;  
  
    n = 0;  
    k = i;  
  
    //Loop-Invariante:  
    While (k != 0) {  
        n += k % 10;  
        k = k / 10;  
    }  
  
    //Postcondition: quersumme(i) == n  
    return n;  
} //quersumme(i) gibt die Quersumme von int i zurück (i >= 0)
```

Loop-Invariante vorgehen (Fortsetzung):

Inv: $\text{quersumme}(i) - \text{quersumme}(k) == n$

- Es fehlt noch etwas!
- Der Parameter von `quersumme()` muss positiv sein:
- $i \geq 0 \ \&\& \ k \geq 0$
- (ein Hint, um darauf zu kommen, kann uns die Precondition geben)

```

public static int compute(int i) {
    // Precondition: i >= 0

    int n;

    int k;

    n = 0;

    k = i;

    //Loop-Invariante:

    While (k != 0) {

        n += k % 10;

        k = k / 10;

    }

    //Postcondition: quersumme(i) == n

    return n;

} //quersumme(i) gibt die Quersumme von int i zurück (i >= 0)

```

Loop-Invariante vorgehen (Fortsetzung):

Inv: $\text{quersumme}(i) - \text{quersumme}(k) == n$

- Es fehlt noch etwas!
- Der Parameter von quersumme muss positiv sein:
- $i \geq 0 \ \&\& \ k \geq 0$
- (ein Hint, um darauf zu kommen, kann uns die Precondition geben)
- Fertige Invariante:
 $\text{quersumme}(i) - \text{quersumme}(k) == n \ \&\& \ i \geq 0 \ \&\& \ k \geq 0$